

ICT1011

Computer Organisation and Architecture

Chapter 3

Instruction Set Architecture

*- Example of a computer system using
the MSP430-*

Chapter Contents

1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

Chapter Contents



<https://ict1011.stevenwong.org/c3t1/>

1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

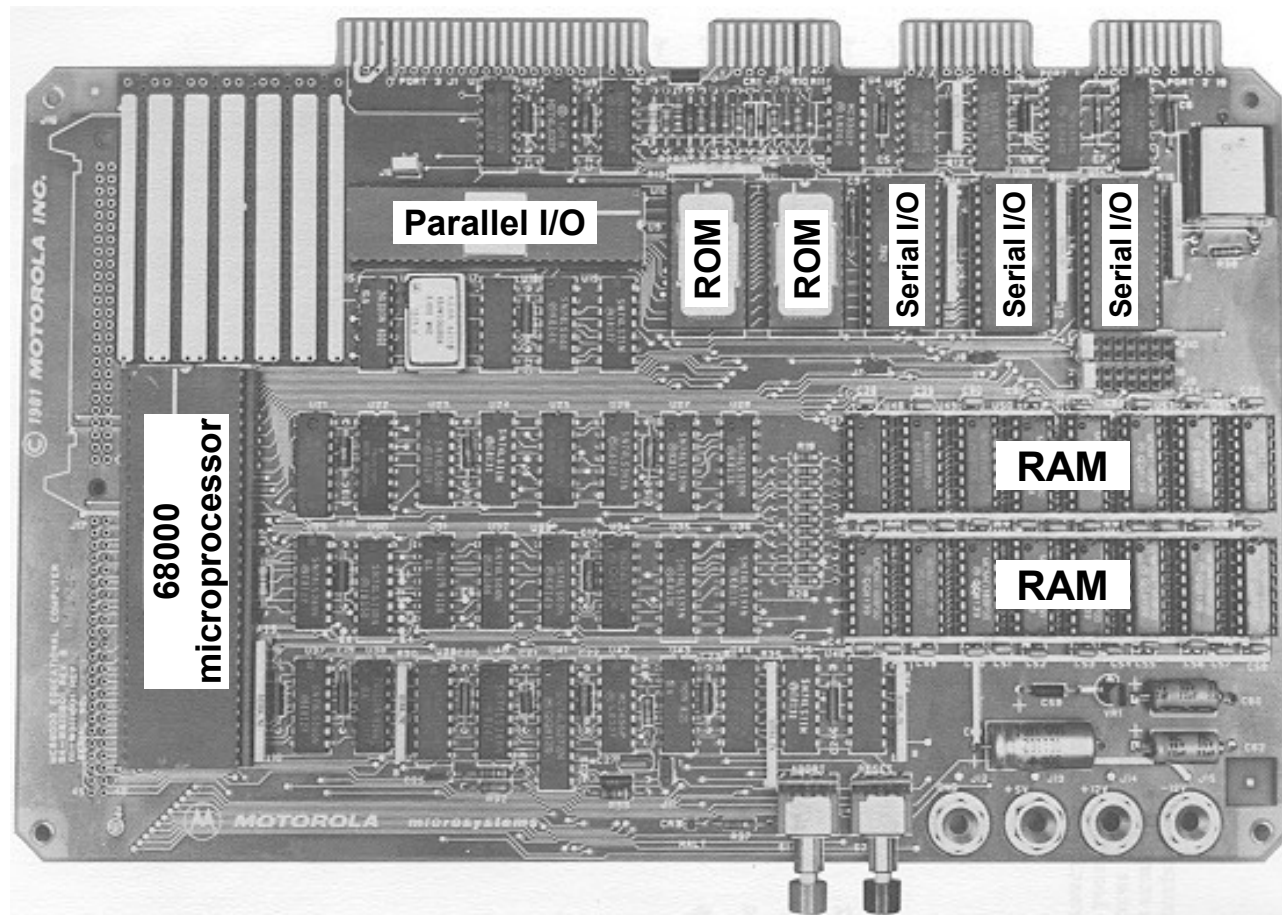
2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

Microprocessor vs. Microcontroller

Microprocessor-based System

- A **microprocessor** (or μP) forms the core of the system with external **memory** and **I/O** support added to provide operational capabilities.

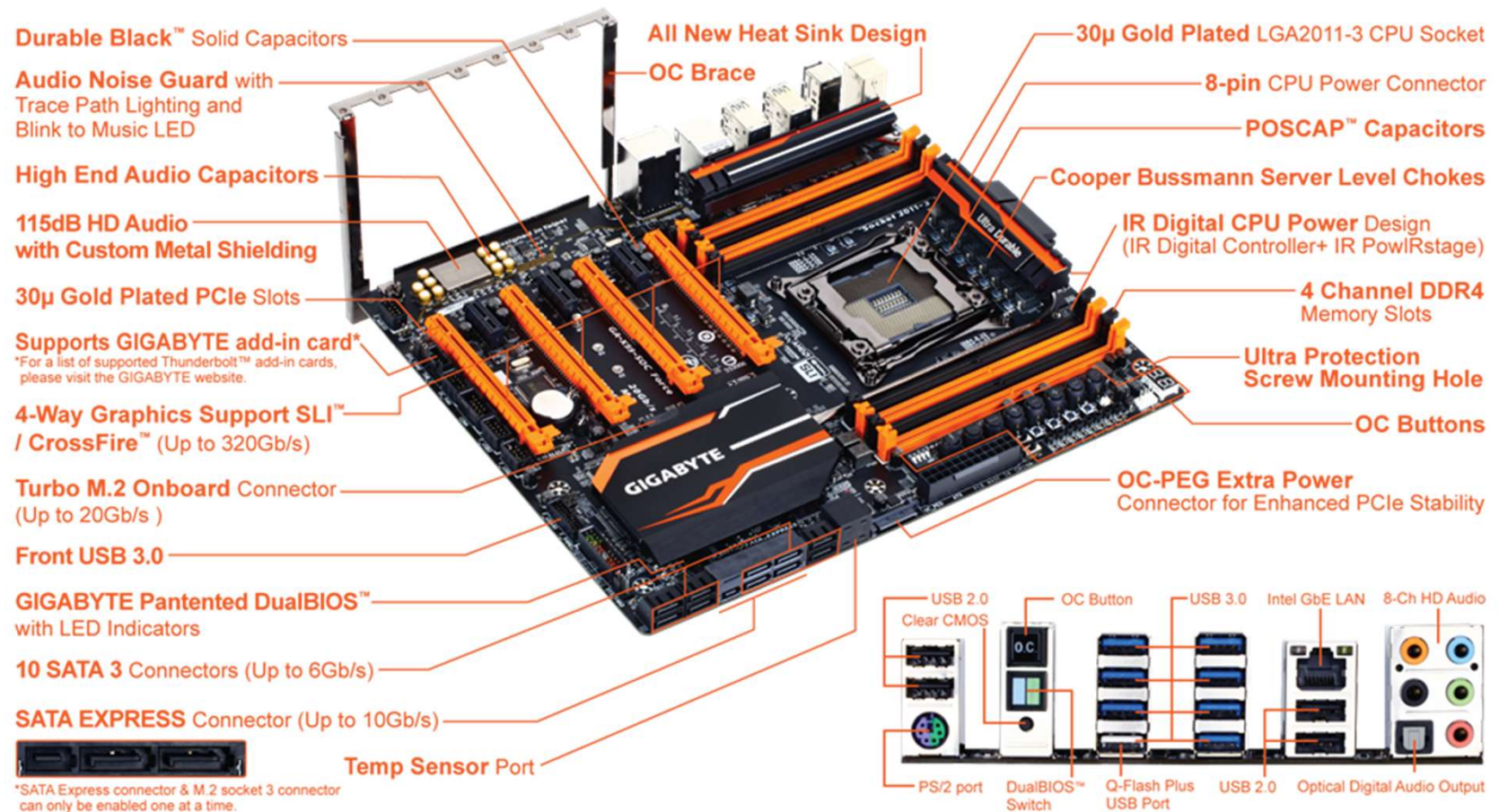


A Basic Motorola 68000 Microprocessor System

Microprocessor vs. Microcontroller

Microprocessor-based System

- Many of the modern day desktop computers are microprocessor-based systems.

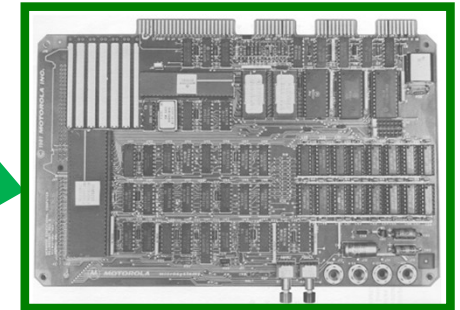
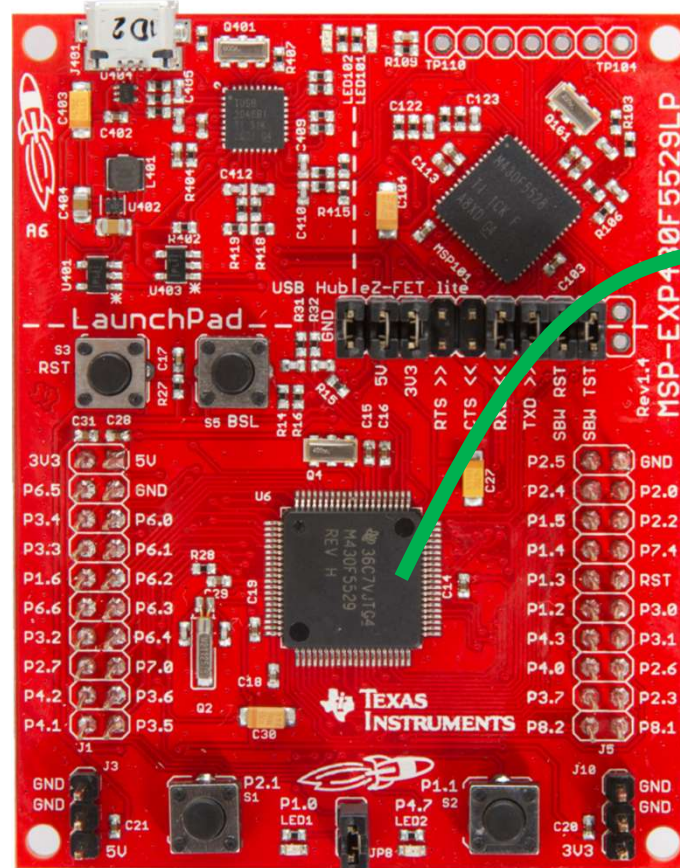


The Gigabyte GA-X99-SOC Force supporting the Intel Core i7 microprocessors

Microprocessor vs. Microcontroller

Microcontroller-based System

- A microcontroller (MCU or μC) has built-in **memory** and **I/O** support, giving rise to a compact operational system.



A single chip has all the equivalent components of a 68000 μP system

A MSP430F5529 microcontroller based computer

Microprocessor vs. Microcontroller

Example of a MCU

MSP430F59xx
Ultra Low Power
16-bit MCU

16MHz



Memory

- RAM
- MPU

Debug

- Real Time JTAG
- Embedded emulation
- Bootstrap Loader

Accelerators

- 32x32 Multiplier
- DMA (3 Ch)
- CRC16
- AES256 Encryption (FR59xx)

Serial Interfaces

- 3 Serial Interfaces (eUSCI)
- 2 UART + IrDA or SPI
- 2 I2C or SPI

Analog

- 12-bit SAR ADC (up to 16 ch)
 - Differential inputs
 - Window comparators
- Comparator (Comp_E)
- Vref (REF_A)

Timers

- Watch Dog Timer (WDT_A)
- Real Time Clock (RTC_B)
- Two 16-bit w/3 CCR (TA0, TA1)
- Two 16-bit w/2 CCR (TA2, TA3)
- One 16-bit w/7 CCR (TB0)

Connectivity

- Up to 40 GPIO (Interrupt/Wake)
- Cap touch IO

Power & Clocking

- Brownout Reset
- Supply Voltage Supervisor (SVS)
- Low Power Vreg (1.5V LDO)
- External Oscillators: LFXT, HFXT
- Internal Oscillators: VLO, DCO ($\pm 2\%$)

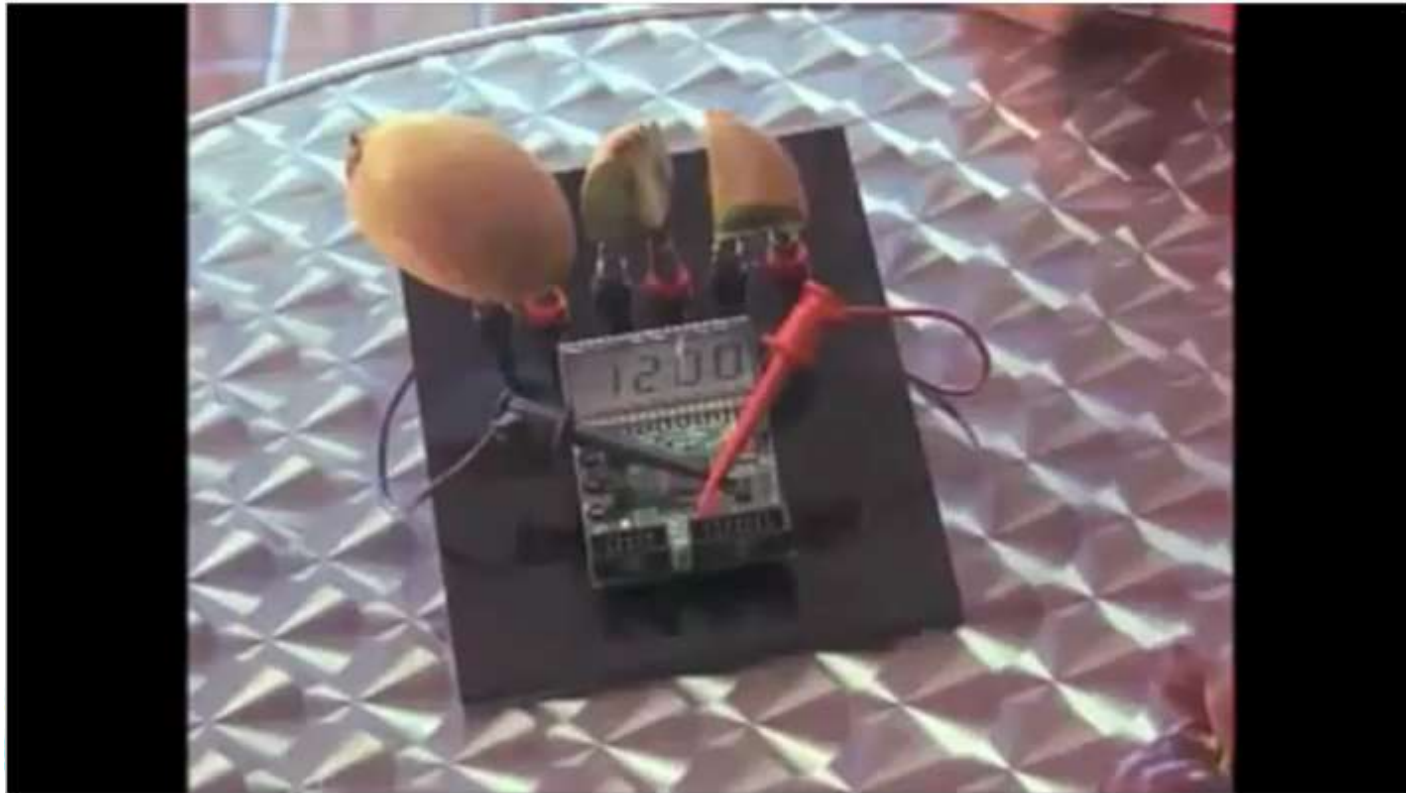
Microcontroller-based System

- Properties of a typical microcontroller (μC) system:
 - **Integration:** Able to implement a whole design onto a single chip.
 - **Cost:** Are usually low-cost devices (a few \$ each);
 - **Clock frequency:** Compared with other devices (microprocessors and DSPs), MCUs use a low clock frequency:
 - MCUs today run up to 100 MHz/100 MIPS (Million Instructions Per Second).
 - **Power consumption:** Low power (battery operation);
 - **Bits:** 4 bits (older devices) to 32 bits devices;
 - **Memory:** Limited available memory, usually less than 1 MByte;
 - **Input/Output (I/O):** Low to high (8 to 150) pin-out count.

Microprocessor vs. Microcontroller

MSP430 – An example of a microcontroller

- **Low power consumption:**
 - 0.1 μA for RAM data retention;
 - 0.8 μA for real-time clock mode operation;
 - 250 $\mu\text{A}/\text{MIPS}$ during active operation.

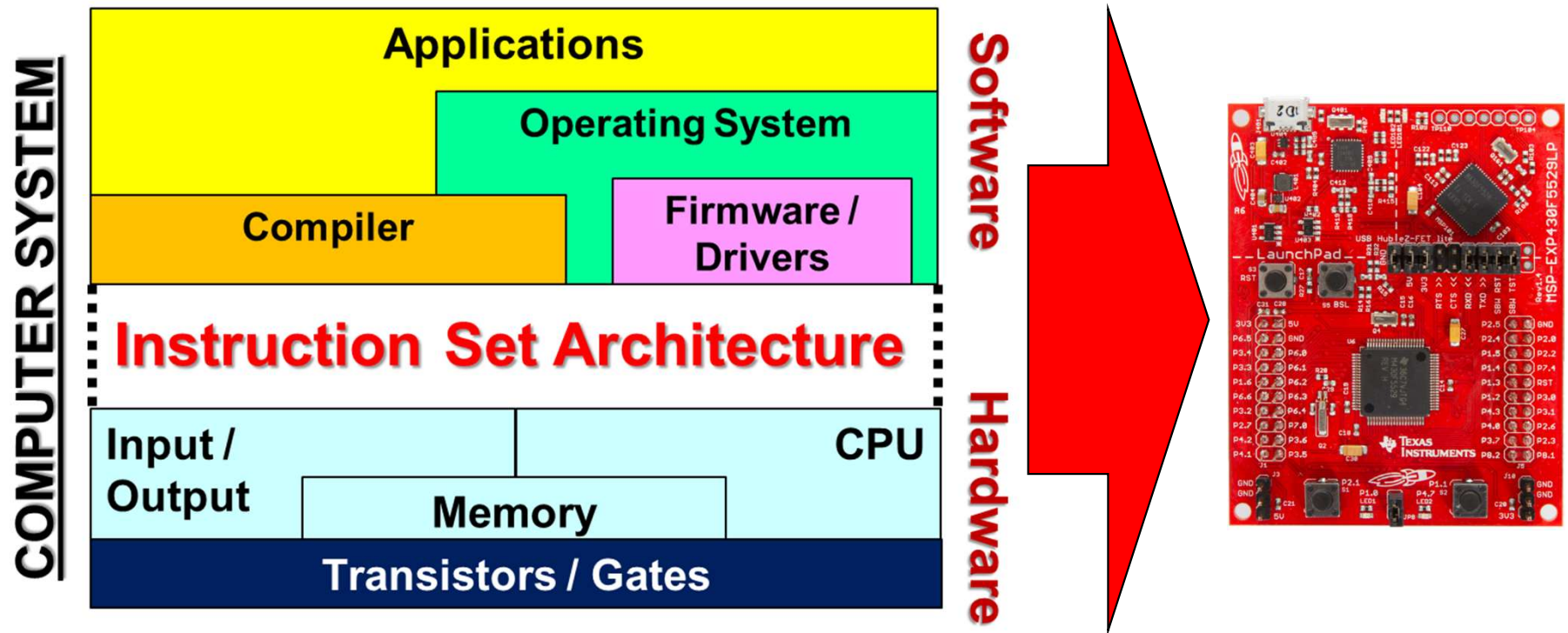


Power a MSP430 “computer” using fruits!!!

Microprocessor vs. Microcontroller

Microcontroller-based System

- Why are we using a microcontroller-based systems for this course???



It is still a computer and representative of generic computers. It is also less complex and is cheap!!!

Note: ICT1011 uses the MSP430 to demonstrate principals of Computer Architecture and Organisation. Thus, not everything on MSP430 is covered in this module. More about the MSP430 and other processors will be covered in other modules!!!

Chapter Contents



<https://ict1011.stevenwong.org/c3t2/>

1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

MSP430 MCU Overview

Microcontrollers (MCU)				Application (MPU)		
MSP430	C2000	Tiva C	Hercules	Sitara	DSP	Multicore
16-bit Ultra Low Power & Cost	Real-time	All-around MCU	32-bit Safety	32-bit Linux Android	16/32-bit All-around DSP	32-bit Massive Performance
MSP430 ULP RISC MCU	Real-time		ARM Cortex-M3 Cortex-R4	ARM Cortex-A8 Cortex-A9	DSP C5000 C6000	- C66 + C66 - A15 + C66 - A8 + C64 - ARM9 + C674
- Low Pwr Mode 0.1 μA 0.5 μA (RTC) - Analog I/F - USB and RF	- Motor Control - Digital Power - Precision	32-bit Float - Nested Vector IntCtrl (NVIC) - Ethernet	- Lock step Dual-core R4 - ECC Memory - SIL3 Certified	\$5 Linux CPU 3D Graphics - PRU-ICSS industrial subsys	- C5000 Low Power DSP 32-bit fix/float C6000 DSP	- Fix or Float - Up to 12 cores 4 A15 + 8 C66x - DSP MMAC's: 352,000
TI-RTOS	TI-RTOS (k)	TI-RTOS	3 rd Party (only)	Linux, Android, TI-RTOS Kernel	C5x: DSP/BIOS C6x: TI-RTOS (k)	Linux TI-RTOS (k)
Flash: 512K FRAM: 64K	Flash	Flash	256K to 3M Flash	L1: 32K x 2 L2: 256K	L1: 32K x 2 L2: 256K	L1: 32K x 2 L2: 1M + 4M
25 MHz			220 MHz	1.35 GHz	800 MHz	1.4 GHz
\$0.25 to \$9.00	\$1.85 to \$20.00	\$1.00 to \$8.00	\$5.00 to \$30.00	\$5.00 to \$25.00	\$2.00 to \$25.00	\$30.00 to \$225.00

CPU Internal Bus

RISC

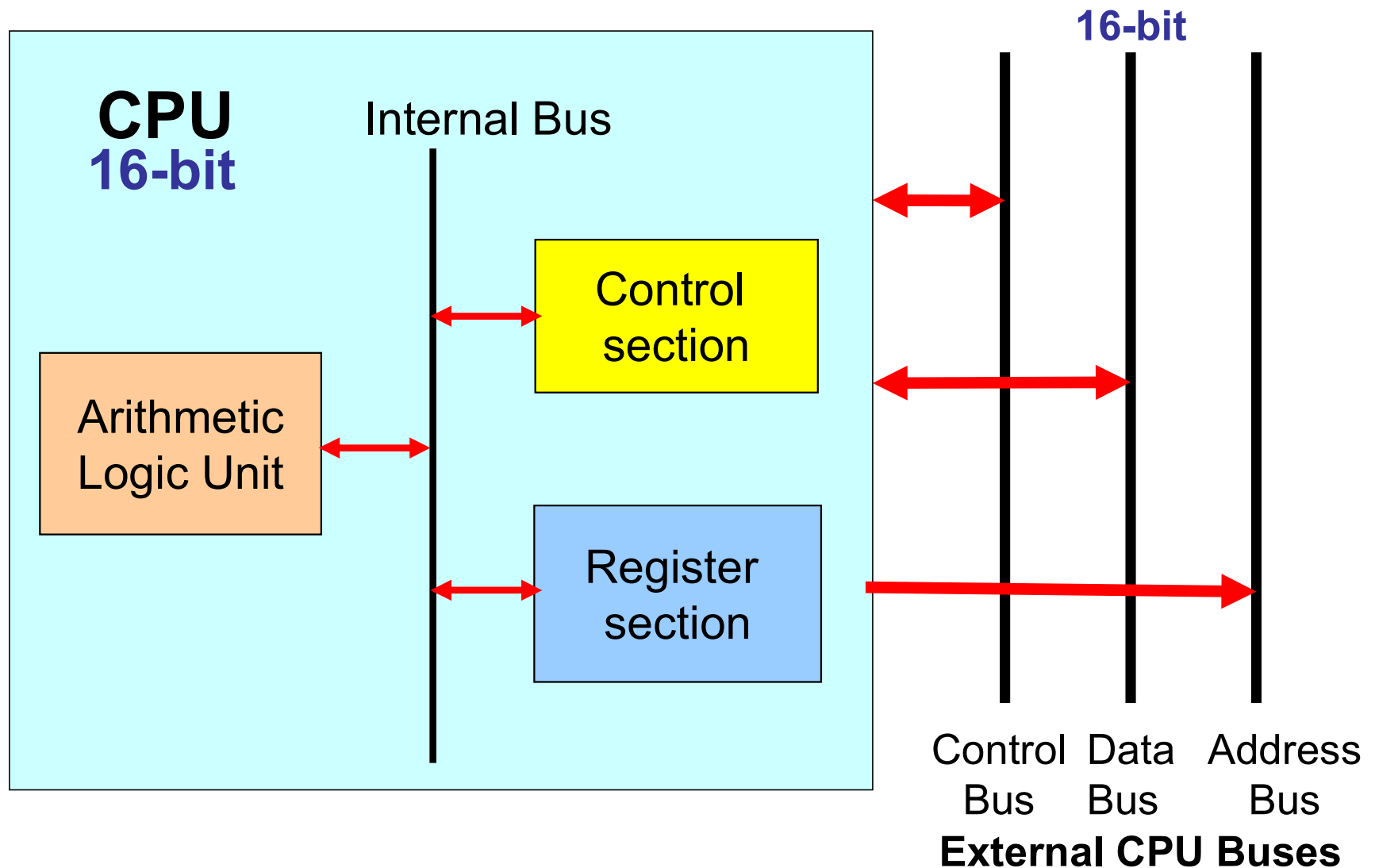
Input / Output

Memory

Clock

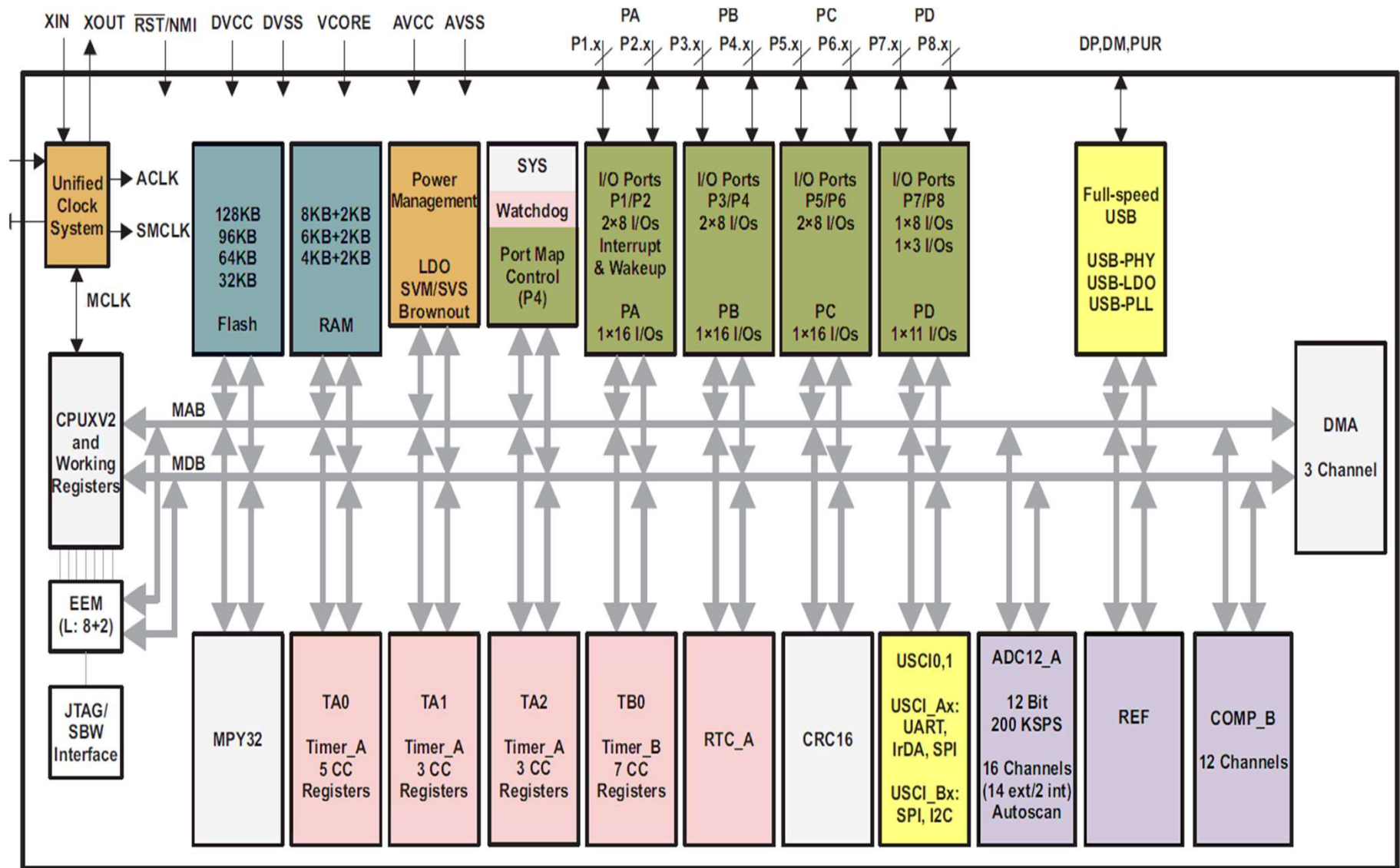
Internal and External Buses

The MSP430 has a 16-bit internal architecture, a 16-bit external data bus.



MSP430 Overview

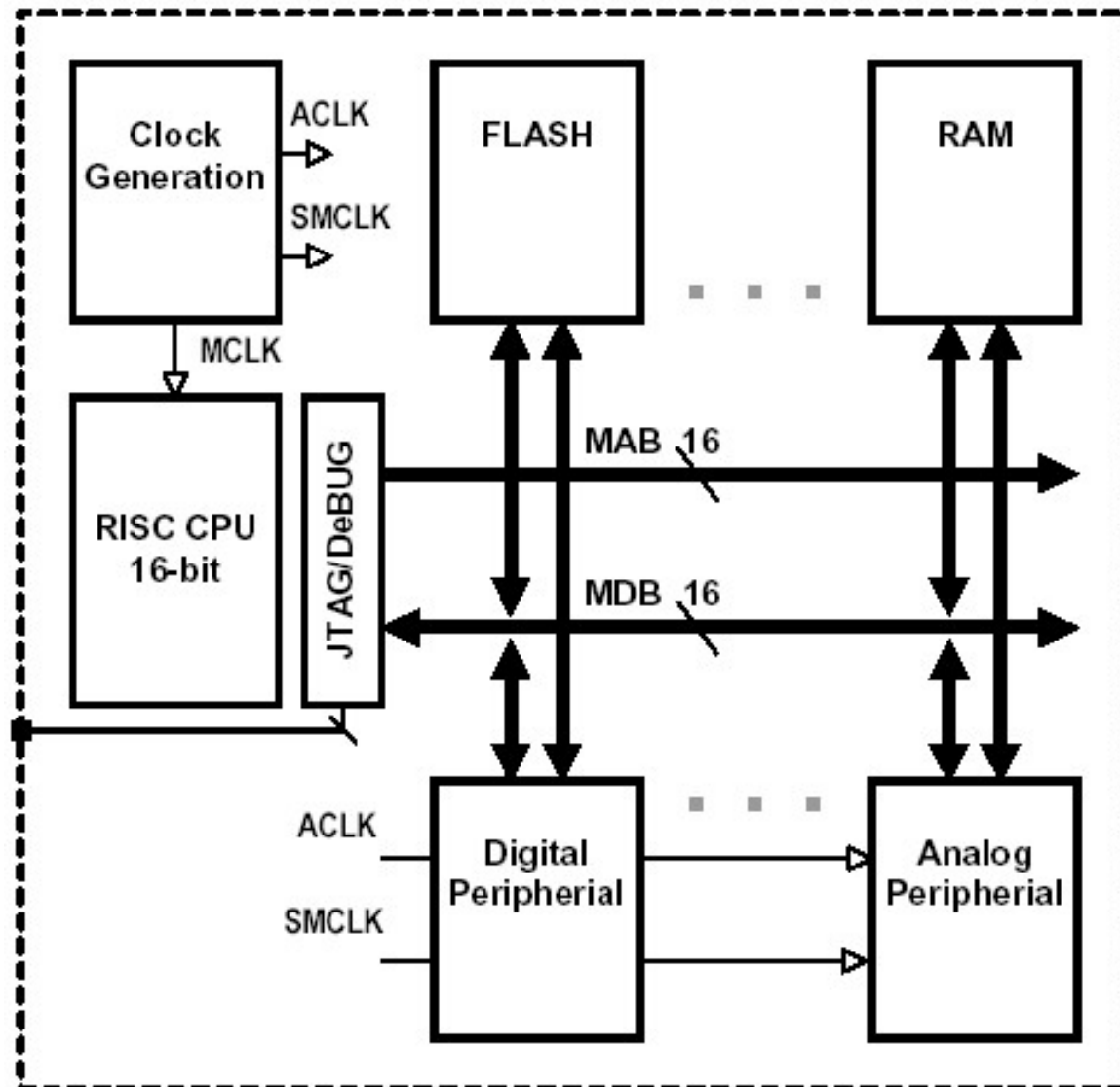
MSP430 Detailed Block Diagram



This is all inside one single MSP430 chip!!!

MSP430 Overview

Simplified Block Diagram



Von-Neumann architecture – common bus to all memory and peripherals

MAB – Memory Address Bus

MDB – Memory Data Bus

Size of MAB (e.g. 16-bits) depends on the device model

This is all inside one single MSP430 chip!!!

Chapter Contents



<https://ict1011.stevenwong.org/c3t3/>

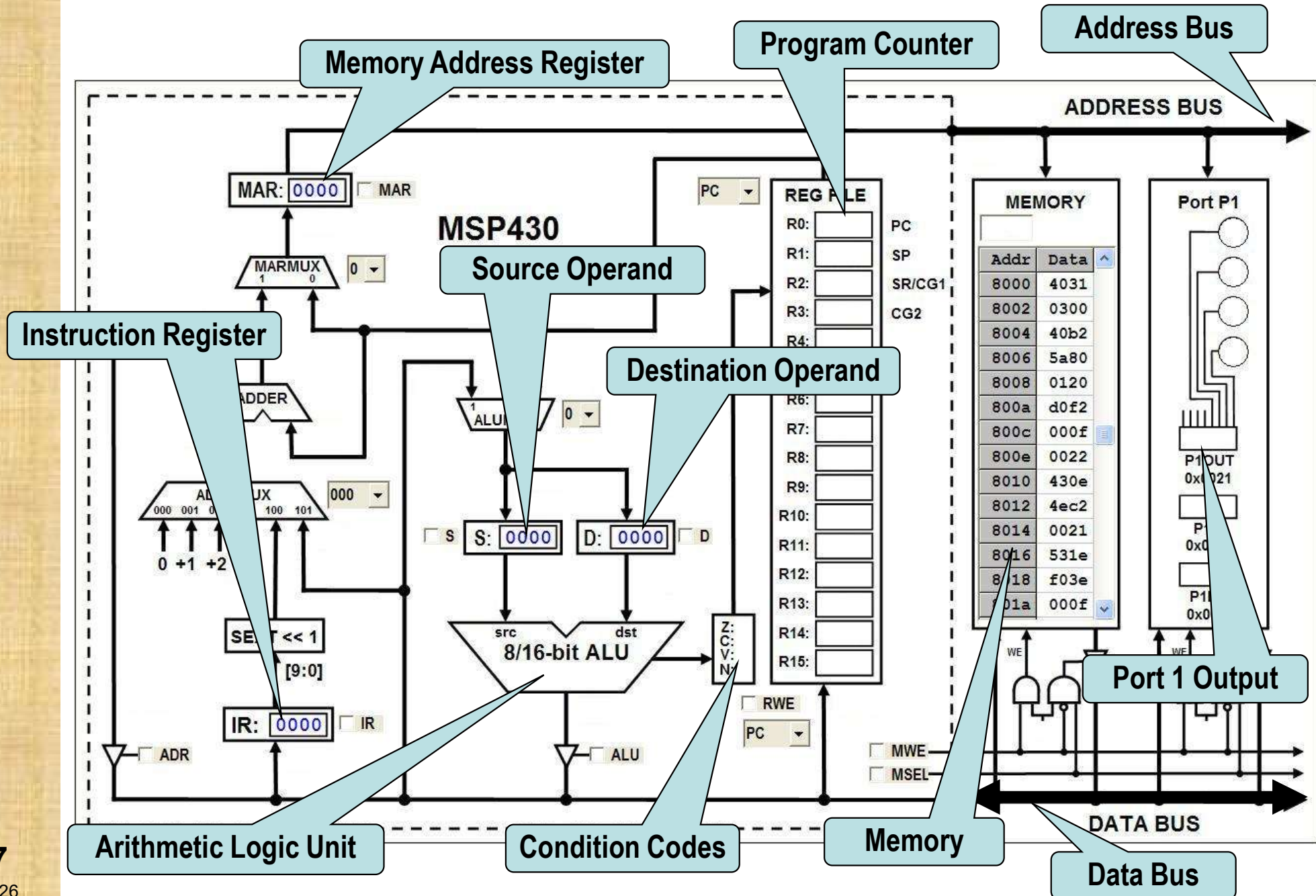
1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

The MSP430 Micro-Architecture



Main Characteristics

16-bit RISC CPU

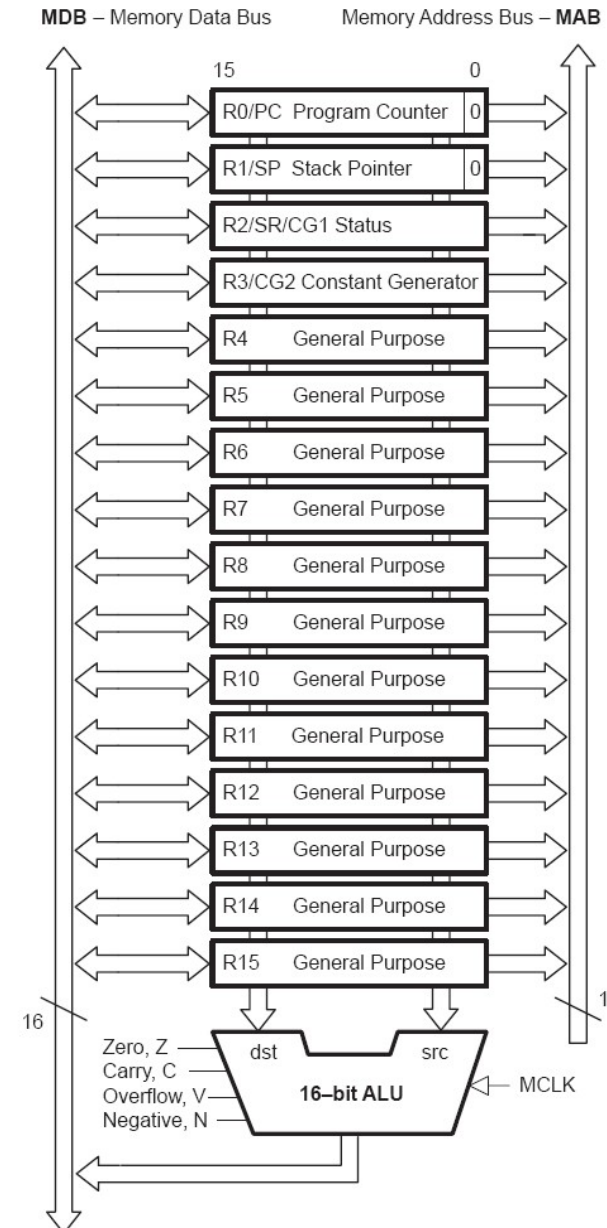
- Compact core design reduces power consumption and cost
- 27 core instructions (*8 Jump, 7 single and 12 double-operand instructions*)
- 7 addressing modes
- 8/16-bit instruction addressing format

Memory architecture

- 16 16-bit registers (*4 dedicated-use and 12 general registers*)
- 16-bit Arithmetic Logic Unit (ALU)
- 16-bit data bus (*8-bit addressability*)
- Supports 8/16-bit peripherals
- Address bus size is dependent on model

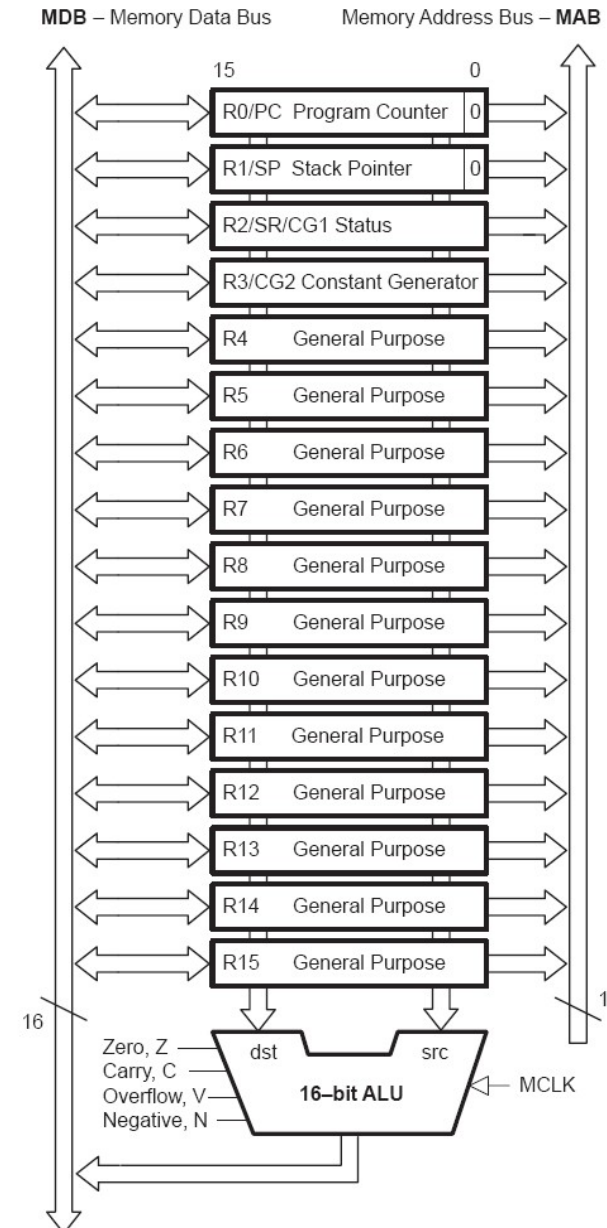
Organisation of the CPU

- Organization of the MSP430 CPU:
 - 16-bit data bus (MDB). In this example, a 16-bit address bus (MAB) is shown.
 - Both registers and memory can be accessed either in word format or in byte format.
 - Allows the direct transfer of data between memory, without passing through the registers.
 - The 16-bit registers can be accessed directly through the instructions, some of which run in a single clock cycle.
 - Some of the constants most used in programming can be obtained from the constant generator.



Organisation of the CPU

- Organization of the MSP430 CPU (continued):
 - The architecture has a 16 bit Arithmetic Logic Unit (ALU).
 - Carrying out operations affects the state of the following flags:
 - Zero (Z);
 - Carry (C);
 - Overflow (V);
 - Negative (N).
 - The MCLK (Master) clock signal drives the CPU.



Chapter Contents



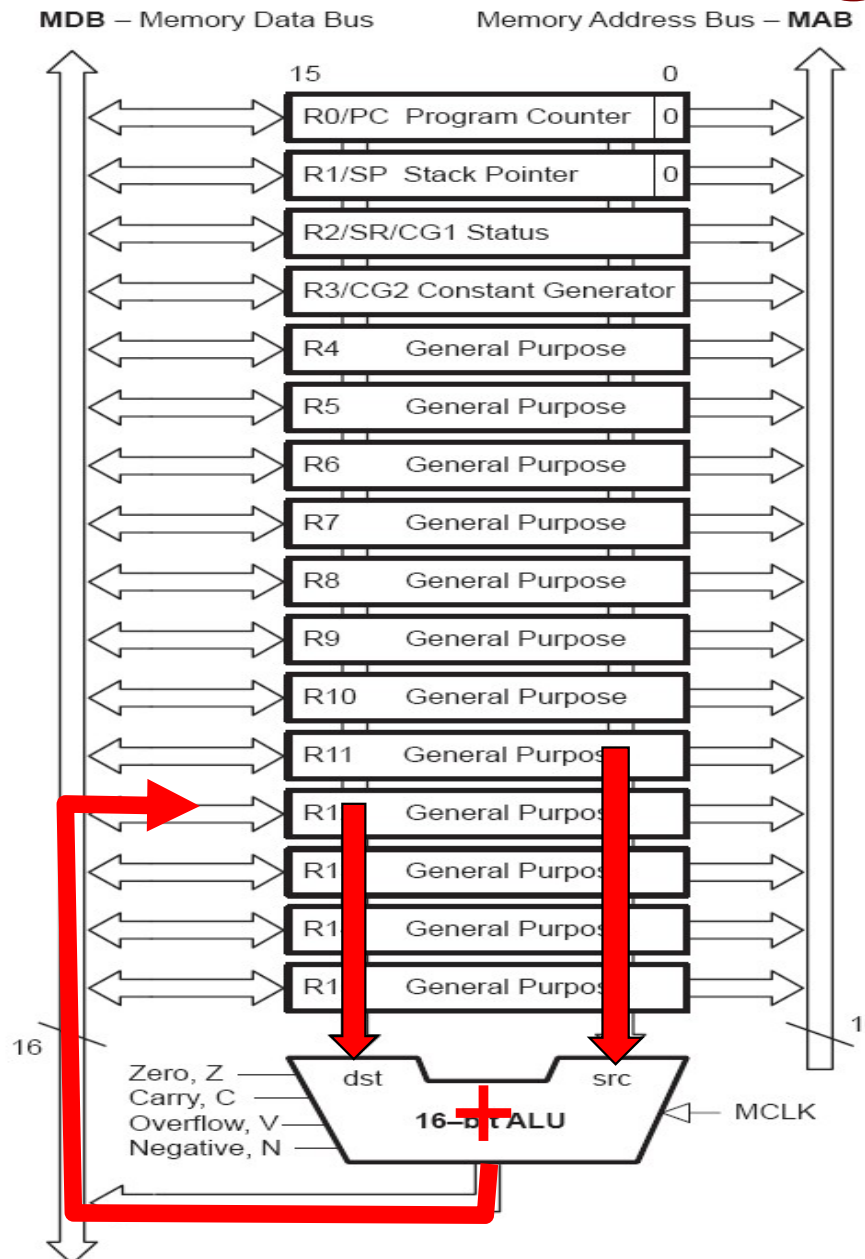
1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

Registers



R0 = Program Counter (PC)

R1 = Stack Pointer (SP)

R2 = Status Register (SR)

R3 / R2 = Constant Generator (CG)

R4 to R15

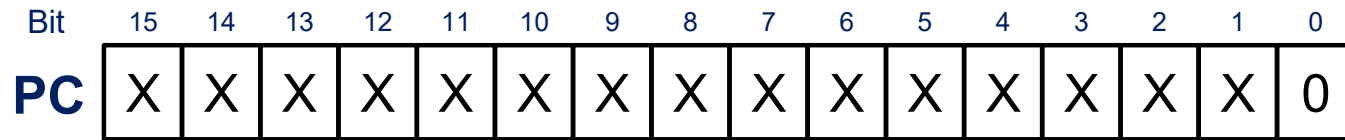
General Purpose Registers (also known as Working Registers)

R4 to R15 are single-cycle, general purpose and identical in all aspects – used for math, storage and addressing modes.

Example:

add.w R11, R12 ;R12=R11+R12

R0 – Program Counter (PC)



X can be 1 or 0

- The 16-bit program counter (PC/R0) points to the next instruction to be executed
- Each instruction uses an even number of bytes (two, four, or six), and the PC is incremented accordingly.
- Instruction accesses in the address space are performed on word boundaries, and the PC is aligned to even addresses
- PC can be addressed by all instructions and all addressing modes
 - `MOV #LABEL,PC ; Branch to address LABEL`
 - `MOV LABEL,PC ; Branch to address contained in LABEL`
 - `MOV @R14,PC ; Branch indirect to address in R14`

MSP430 CPU ISA - Registers

R1 – Stack Pointer (SP)



- The stack pointer (SP/R1) is used by the CPU to store the return addresses of subroutine calls and interrupts. It also can be used to store local data.
- It uses a pre-decrement, post-increment scheme. SP always point to a RAM location and is always even

Examples

- PUSH #0123h ; Put 0123h onto Top of (Stack TOS)
- POP R8 ; R8 = 0123h

Addr	Content
1008h	
1006h	
1004h	← SP
1002h	
1000h	

Addr	Content
1008h	
1006h	
1004h	
1002h	0123h ← SP
1000h	

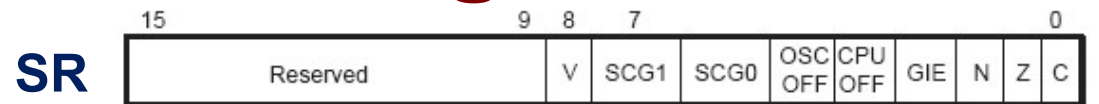
PUSH #0123h

Addr	Content
1008h	
1006h	
1004h	← SP
1002h	0123h
1000h	

POP R8
R8 = 0123h

MSP430 CPU ISA - Registers

R2 – Status Register



- The status register (SR/R2), used as a source or destination register, can be used in the register mode only addressed with word instructions.
- The remaining combinations of addressing modes are used to support the constant generator.

We will cover address modes and the examples of the V,N,Z,C flags later!!!

Bit	Description
V	Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range. ADD (. B) , ADDC (. B) Set when: Positive + Positive = Negative Negative + Negative = Positive, otherwise reset SUB (. B) , SUBC (. B) , CMP (. B) Set when: Positive – Negative = Negative Negative – Positive = Positive, otherwise reset
SCG1	System clock generator 1. This bit, when set, turns off the SMCLK.
SCG0	System clock generator 0. This bit, when set, turns off the DCO dc generator, if DCOCLK is not used for MCLK or SMCLK.
OSCOFF	Oscillator Off. This bit, when set, turns off the LFXT1 crystal oscillator, when LFXT1CLK is not use for MCLK or SMCLK
CPUOFF	CPU off. This bit, when set, turns off the CPU.
GIE	General interrupt enable. This bit, when set, enables maskable interrupts. When reset, all maskable interrupts are disabled.
N	Negative bit. This bit is set when the result of a byte or word operation is negative and cleared when the result is not negative. Word operation: N is set to the value of bit 15 of the result Byte operation: N is set to the value of bit 7 of the result
Z	Zero bit. This bit is set when the result of a byte or word operation is 0 and cleared when the result is not 0.
C	Carry bit. This bit is set when the result of a byte or word operation produced a carry and cleared when no carry occurred.

R2/R3 – Constant Generators

- R2/R3: Constant Generator Registers (CG1/CG2):
 - Depending of the source-register addressing modes (As) value, six constants can be generated without code word or code memory access to retrieve them.
 - This is a very powerful feature which allows the implementation of emulated instructions, for example, instead of implement a core instruction for an increment the constant generator is used.

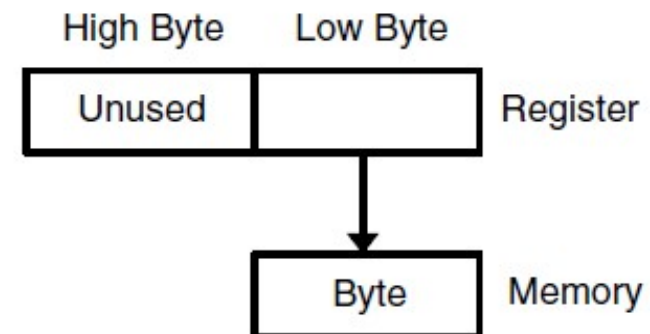
Register	As	Constant	Remarks
R2	00	-	Register mode
R2	01	(0)	Absolute mode
R2	10	00004h	+4, bit processing
R2	11	00008h	+8, bit processing
R3	00	00000h	0, word processing
R3	01	00001h	+1
R3	10	00002h	+2, bit processing
R3	11	0FFFFh	-1, word processing

Remember that memory access is slow. It is better to use the registers to store commonly used constants.

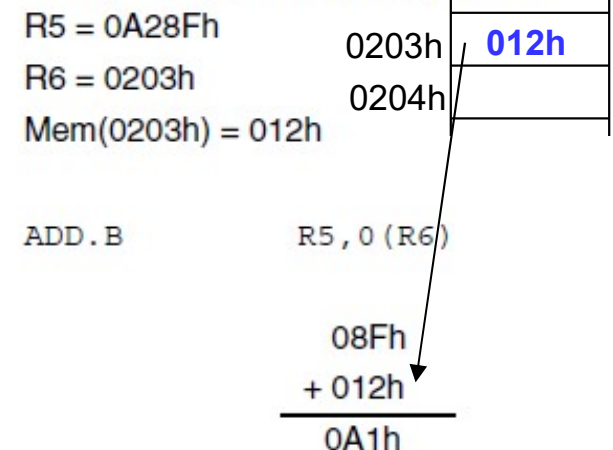
General Purpose Registers

- R4-R15: General-purpose registers:
 - The general purpose registers R4 to R15 can be used as data registers, data pointers and indices. They can be accessed either as a byte or as a word;
 - These registers support operations on words or bytes;
 - In the example to the right, the contents of the least significant byte of register R5 (0x8F) is added to the contents of the memory address pointed to by the register R6 (0x12h);

Register-Byte Operation



Example Register-Byte Operation



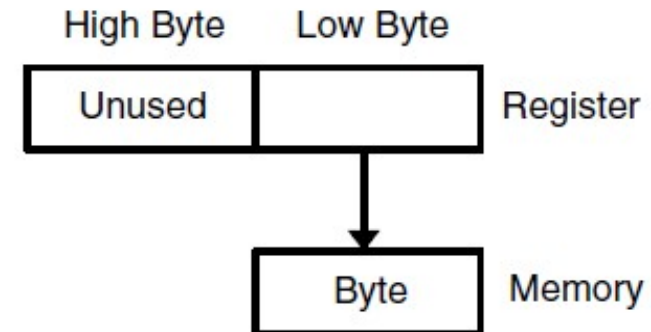
Mem (0203h) = 0A1h

C = 0, Z = 0, N = 1

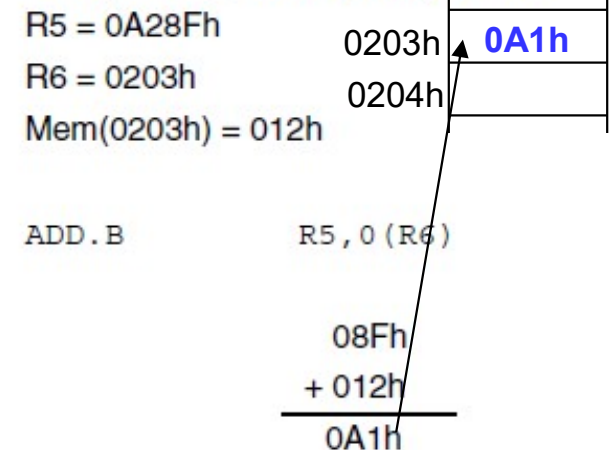
General Purpose Registers

- R4-R15: General-purpose registers:
 - The contents of the memory address is updated with the result of the operation (0xA1);
 - The status flags of the CPU in the SR are updated after the execution of the instruction.

Register-Byte Operation



Example Register-Byte Operation



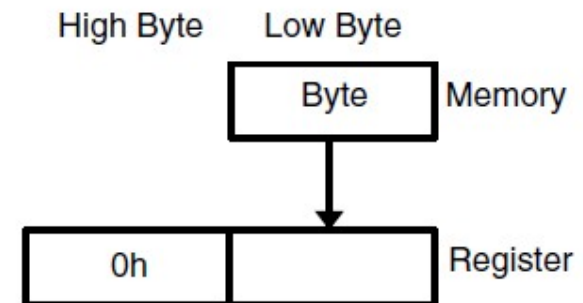
Mem (0203h) = 0A1h

C = 0, Z = 0, N = 1

General Purpose Registers

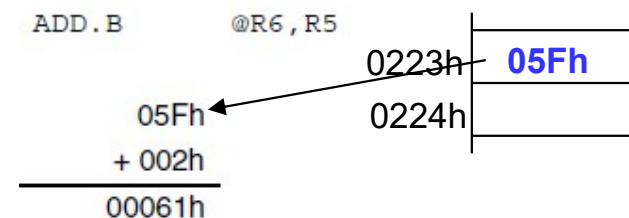
- R4-R15: General-purpose registers:
 - The contents of the memory address pointed to by R6 (0x5F) is added to the contents of the least significant byte of register R5 (0x02);
 - The result of the operation (0x61) is stored in the least significant byte of register R5;
 - Meanwhile, the most significant byte of register R5 is set to zero;
 - The system flags in SR are updated in accordance with the result.

Byte-Register Operation



Example Byte-Register Operation

R5 = 01202h
 R6 = 0223h
 Mem(0223h) = 05Fh



R5 = 00061h
 C = 0, Z = 0, N = 0

Chapter Contents



<https://ict1011.stevenwong.org/c3t5/>

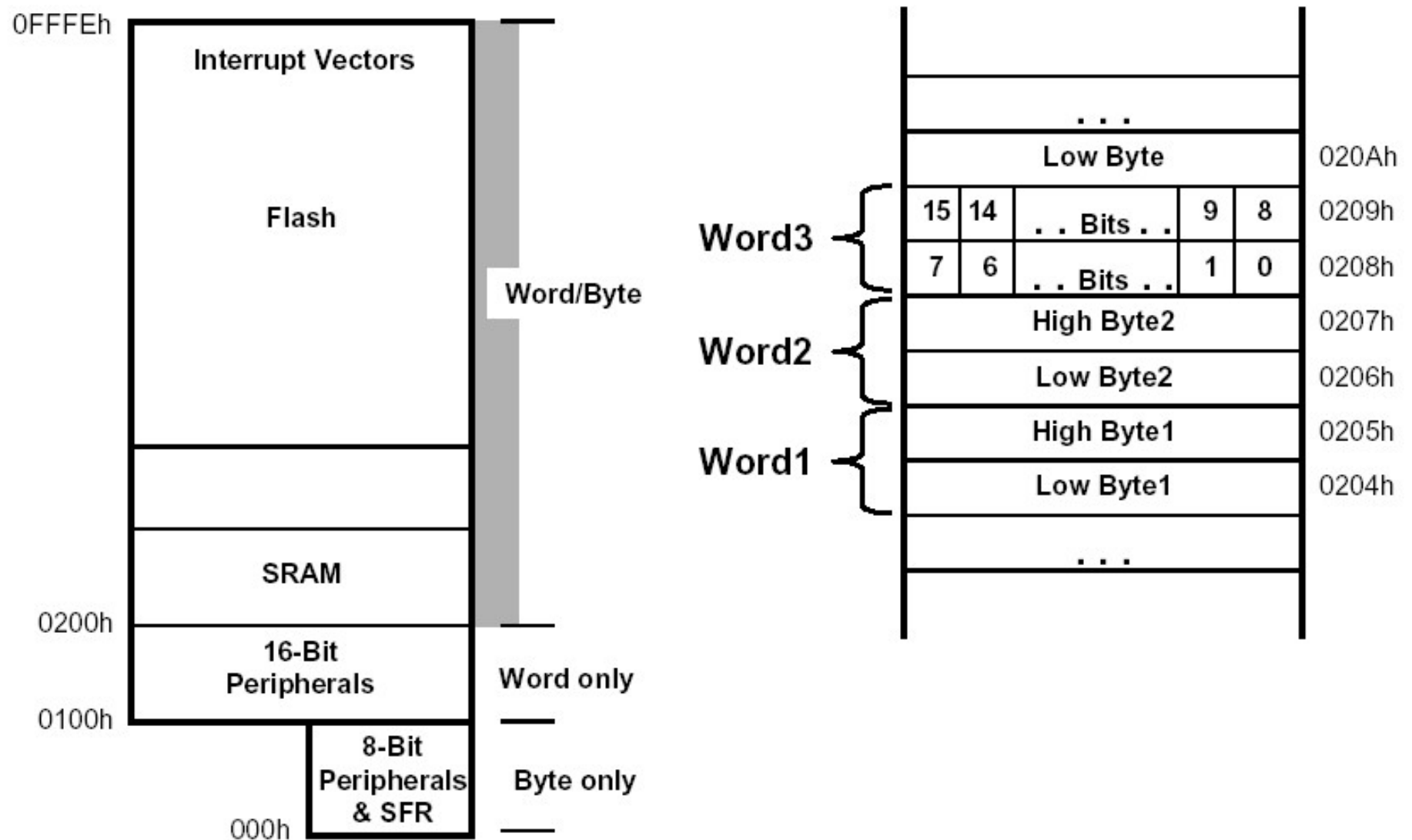
1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- **Memory Organisation**
- Machine Code & Assembly Instructions
- Addressing Modes

Example of the Main Memory



Example of a memory map

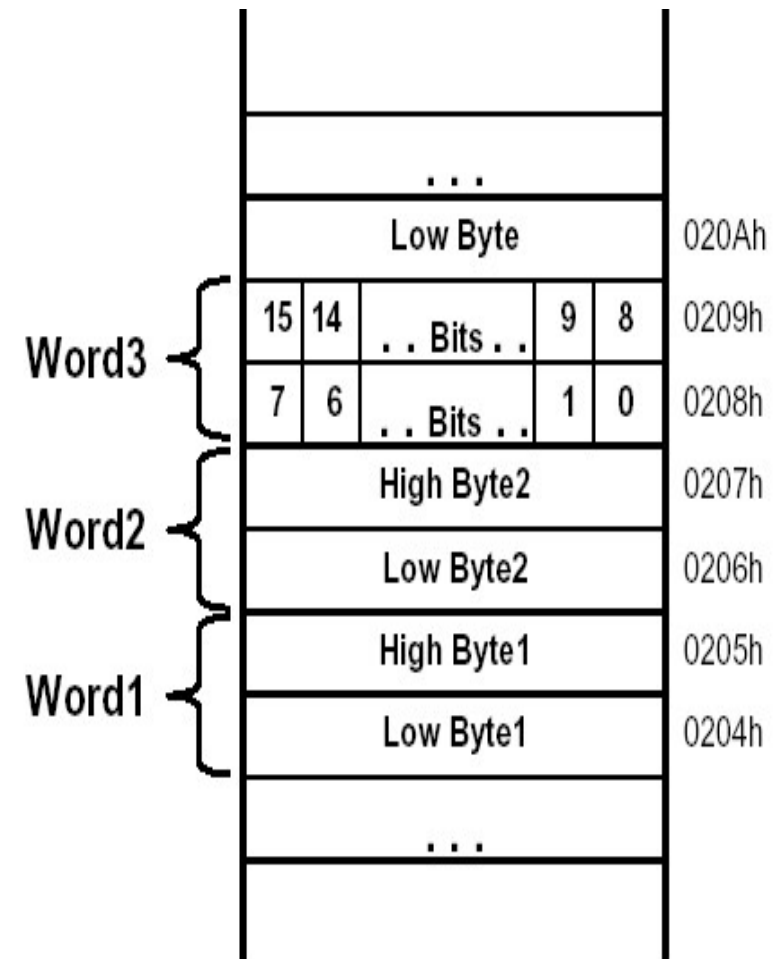
Memory Organisation

- Word alignment**

- Bytes are located at even or odd addresses
- Words are only located at even addresses

- Endianess (little-endian)**

- When using word instructions, only even addresses may be used. The low byte of a word is always an even address.
- The high byte is at the next odd address.
- For example, if a data word is located at address xxx4h, then the low byte of that data word is located at address xxx4h, and the high byte of that word is located at address xxx5h.



Memory Organisation

An Example

Assume value of R8 and R9 is initialised to 0

Retrieving a byte from memory

e.g. MOV.B &3000h, R8

Register R8 will have a value of H'12.

Retrieving a word from memory

e.g. MOV.W &3000h, R9

Register R9 will have a value of H'3412. (MOV.W &3001h, R9 will produce the same result)

Address	Content
0x3004	H' 9A
0x3003	H' 78
0x3002	H' 56
0x3001	H' 34
0x3000	H' 12

There is many correct notations to represent hexadecimal numbers:
e.g. H'12AF, 0x12AF, 12AFh, 12AF₁₆

MSP430 CPU ISA

Example of the Main Memory

		MSP430F5522 MSP430F5521 MSP430F5513	MSP430F5525 MSP430F5524 MSP430F5515 MSP430F5514	MSP430F5527 MSP430F5526 MSP430F5517	MSP430F5529 MSP430F5528 MSP430F5519
Memory (flash) Main: interrupt vector	Total Size	32 KB 00FFFFh–00FF80h	64 KB 00FFFFh–00FF80h	96 KB 00FFFFh–00FF80h	128 KB 00FFFFh–00FF80h
Main: code memory	Bank D	N/A	N/A	N/A	32 KB 0243FFh–01C400h
	Bank C	N/A	N/A	32 KB 01C3FFh–014400h	32 KB 01C3FFh–014400h
	Bank B	15 KB 00FFFFh–00C400h	32 KB 0143FFh–00C400h	32 KB 0143FFh–00C400h	32 KB 0143FFh–00C400h
	Bank A	17 KB 00C3FFh–008000h	32 KB 00C3FFh–004400h	32 KB 00C3FFh–004400h	32 KB 00C3FFh–004400h
RAM	Sector 3	2 KB ⁽²⁾ 0043FFh–003C00h	N/A	N/A	2 KB 0043FFh–003C00h
	Sector 2	2 KB ⁽³⁾ 003BFFh–003400h	N/A	2 KB 003BFFh–003400h	2 KB 003BFFh–003400h
	Sector 1	2 KB 0033FFh–002C00h	2 KB 0033FFh–002C00h	2 KB 0033FFh–002C00h	2 KB 0033FFh–002C00h
	Sector 0	2 KB 002BFFh–002400h	2 KB 002BFFh–002400h	2 KB 002BFFh–002400h	2 KB 002BFFh–002400h
USB RAM ⁽⁴⁾	Sector 7	2 KB 0023FFh–001C00h	2 KB 0023FFh–001C00h	2 KB 0023FFh–001C00h	2 KB 0023FFh–001C00h
Information memory (flash)	Info A	128 B 0019FFh–001980h	128 B 0019FFh–001980h	128 B 0019FFh–001980h	128 B 0019FFh–001980h
	Info B	128 B 00197Fh–001900h	128 B 00197Fh–001900h	128 B 00197Fh–001900h	128 B 00197Fh–001900h
	Info C	128 B 0018FFh–001880h	128 B 0018FFh–001880h	128 B 0018FFh–001880h	128 B 0018FFh–001880h
	Info D	128 B 00187Fh–001800h	128 B 00187Fh–001800h	128 B 00187Fh–001800h	128 B 00187Fh–001800h
Bootstrap loader (BSL) memory (flash)	BSL 3	512 B 0017FFh–001600h	512 B 0017FFh–001600h	512 B 0017FFh–001600h	512 B 0017FFh–001600h
	BSL 2	512 B 0015FFh–001400h	512 B 0015FFh–001400h	512 B 0015FFh–001400h	512 B 0015FFh–001400h
	BSL 1	512 B 0013FFh–001200h	512 B 0013FFh–001200h	512 B 0013FFh–001200h	512 B 0013FFh–001200h
	BSL 0	512 B 0011FFh–001000h	512 B 0011FFh–001000h	512 B 0011FFh–001000h	512 B 0011FFh–001000h
Peripherals	Size	4 KB 000FFFh–0h	4 KB 000FFFh–0h	4 KB 000FFFh–0h	4 KB 000FFFh–0h

Detailed memory map depends on the device model.

MSP430 CPU ISA

Example of the Main Memory

'F5529 Memory Map

Main Flash

- ♦ 512 byte segments
- ♦ Start address moves according to RAM

Flash

Bytes

128K

MSP430 Memory

- ♦ Unified memory map (program or data)
- ♦ Absolutely no paging

Info Memory

- ♦ Use for your own calibration data, etc.
- ♦ 4 segments (A-D)
- ♦ 128 byte segments

0xFFFF

0xFF80

0x4400

0x2400

0x01C0

0x0000

INT Vectors

80

RAM

8K

USB RAM

2K

Info Memory

512

Boot Loader

2K

Peripherals

4K

RAM

- ♦ Always a contig. block
- ♦ If enabled, USB port uses first 2K
- ♦ RAM segments can be powered down

Device Descriptors (TLV)

- ♦ Factory calibration data, periph support,...
- ♦ Found in peripherals (at 0x1A00)

Boot Loader (BSL)

- ♦ Program Flash/RAM with serial (slau319)
- ♦ Password protected
- ♦ 512 byte segments

Chapter Contents



1. Examples of Computers

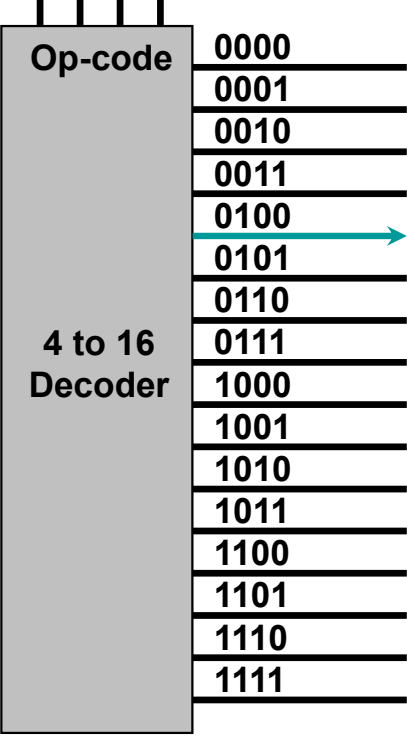
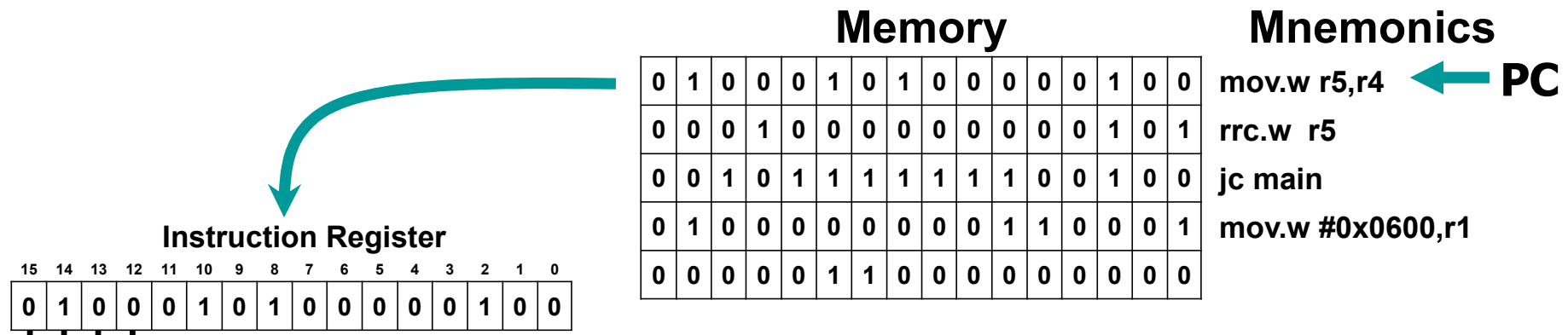
- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

MSP430 CPU ISA

MSP430 Instructions Overview



Op-code	Instruction	Format
0000	Undefined	Single Operand
0001	RCC, SWPB, RRA, SXT, PUSH, CALL, RETI	
0010	JNE, JEQ, JNC, JC	Jumps
0011	JN, JGE, JL, JMP	
0100	MOV	Double Operand
0101	ADD	
0110	ADDC	
0111	SUBC	
1000	SUB	
1001	CMP	
1010	DADD	
1011	BIT	
1100	BIC	
1101	BIS	
1110	XOR	
1111	AND	

MSP430 Instructions Data Types

The MSP430 instruction set supports 3 data types:

- Bit

- Bytes (8 bits)

- Words (16 bits)

} **Basic data types**

Example from Lab 1:

XOR.B #01h, &P1OUT ; (i) This XOR the contents at the address
; P1OUT with the BYTE value 01_{16}
; (ii) The results are stored back at address
; P1OUT

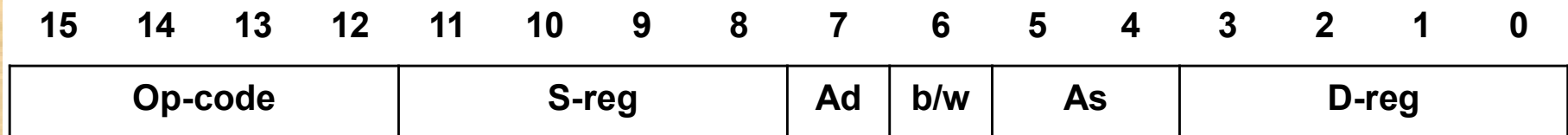
MOV.W #60000, R15 ; This stores a WORD value of 60000_{10} into
; Register R15

MSP430 Instructions

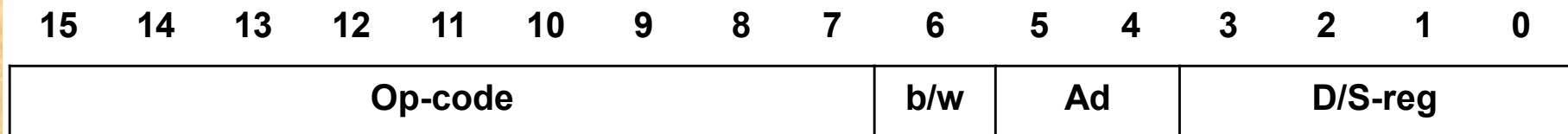
- 27 core instructions
- There are three formats used to encode instructions for processing by the CPU core
 - Double operand
 - Single operand
 - Jumps
- The instructions for double and single operands, depend on the suffix used, (**.w**) word or (**.b**) byte
- These suffixes allow word or byte data access
- If the suffix is ignored, the instruction processes word data by default

MSP430 Instruction Formats

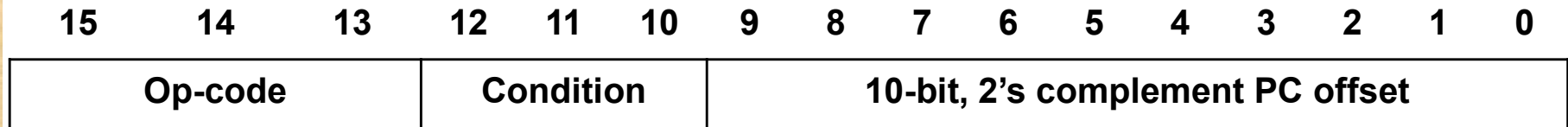
- Format I: Instructions with two operands:



- Format II: Instruction with one operand:



- Format III: Jump instructions:



MSP430 CPU ISA - Instructions

Format I: Double Operand Instructions

- 12 double operand instructions:

Mnemonic	Operation	Description
<i>Arithmetic instructions</i>		
ADD (.B or .W) src,dst	src+dst→dst	Add source to destination
ADDC (.B or .W) src,dst	src+dst+C→dst	Add source and carry to destination
DADD (.B or .W) src,dst	src+dst+C→dst (dec)	Decimal add source and carry to destination
SUB (.B or .W) src,dst	dst+.not.src+1→dst	Subtract source from destination
SUBC (.B or .W) src,dst	dst+.not.src+C→dst	Subtract source and not carry from destination
<i>Logical and register control instructions</i>		
AND (.B or .W) src,dst	src.and.dst→dst	AND source with destination
BIC (.B or .W) src,dst	.not.src.and.dst→dst	Clear bits in destination
BIS (.B or .W) src,dst	src.or.dst→dst	Set bits in destination
BIT (.B or .W) src,dst	src.and.dst	Test bits in destination
XOR (.B or .W) src,dst	src.xor.dst→dst	XOR source with destination
<i>Data instructions</i>		
CMP (.B or .W) src,dst	dst-src	Compare source to destination
MOV (.B or .W) src,dst	src→dst	Move source to destination

MSP430 CPU ISA - Instructions

Format I: Double Operand Instructions

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-reg			Ad	b/w	As	D-reg					

- The source and destination of the data operated by an instruction are defined by the following fields:
 - src**: source operand address, as defined in As and S-reg;
 - dst**: destination operand address, as defined in Ad and D-reg;
 - As**: addressing bits used to define the addressing mode used by the source operand;
 - S-reg**: register used by the source operand;
 - Ad**: Addressing bits used to define the addressing mode used by the destination operand;
 - D-reg**: register used by the destination operand;
 - B/W**: word or byte access definition bit. (0:16-bits, 1: 8-bits)

MSP430 CPU ISA - Instructions

Example: Double Operand

- Copy the contents of a register to another register
 - Assembly: `mov.w r5,r4`
 - Instruction code: `0x4504`

<u>Op-code</u> <i>mov</i>	<u>S-reg</u> <i>r5</i>	<u>Ad</u> <i>Register</i>	<u>b/w</u> <i>16-bits</i>	<u>As</u> <i>Register</i>	<u>D-reg</u> <i>r4</i>
0 1 0 0	0 1 0 1	0	0	0 0	0 1 0 0

- One word instruction
- The instruction instructs the CPU to copy the 16-bit 2's complement number in register `r5` to register `r4`

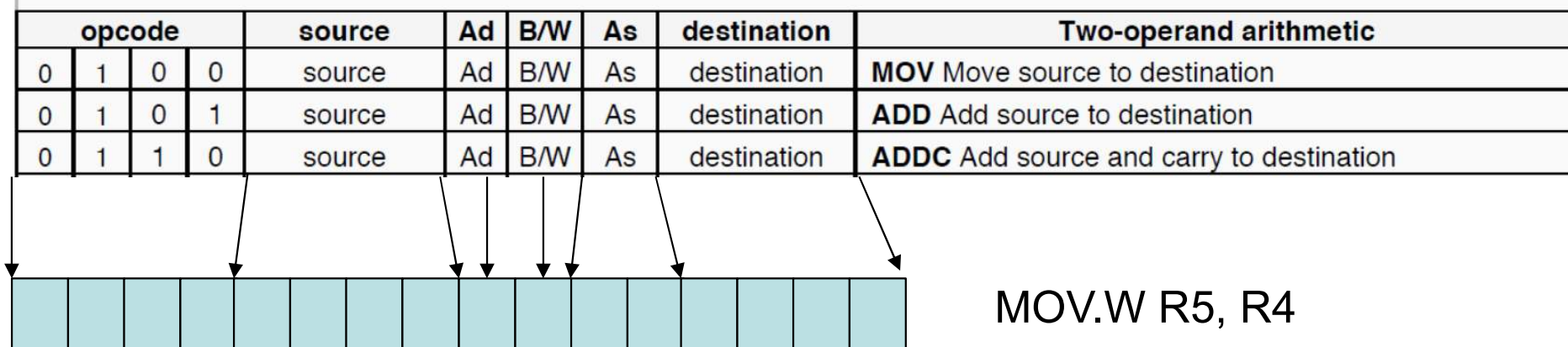
MSP430 Quick Reference Chart

Available in xSITE under the [Contents][Resources] tab

MSP430 Addressing Modes

As	Ad	d/s	Register	Syntax	Description
00	0	ds	$n \neq 3$	Rn	Register direct. The operand is the contents of Rn. $A_d=0$
01	1	ds	$n \neq 0, 2, 3$	x(Rn)	Indexed. The operand is in memory at address $Rn+x$.
10	-	s	$n \neq 0, 2, 3$	@Rn	Register indirect. The operand is in memory at the address held in Rn.
11	-	s	$n \neq 0, 2, 3$	@Rn+	Indirect auto-increment. As above, then the register is incremented by 1 or 2.
Addressing modes using R0 (PC)					
01	1	ds	0 (PC)	LABEL	Symbolic. x(PC) The operand is in memory at address PC+x.
11	-	s	0 (PC)	#x	Immediate. @PC+ The operand is the next word in the instruction stream.
Addressing modes using R2 (SR) and R3 (CG), special-case decoding					
01	1	ds	2 (SR)	&LABEL	Absolute. The operand is in memory at address x.
10	-	s	2 (SR)	#4	Constant. The operand is the constant 4.
11	-	s	2 (SR)	#8	Constant. The operand is the constant 8.
00	-	s	3 (CG)	#0	Constant. The operand is the constant 0.
01	-	s	3 (CG)	#1	Constant. The operand is the constant 1. There is no index word.
10	-	s	3 (CG)	#2	Constant. The operand is the constant 2.
11	-	s	3 (CG)	#-1	Constant. The operand is the constant -1.

B/W: word or byte access definition bit. (0:16-bits, 1: 8-bits)



MSP430 CPU ISA - Instructions

Format II: Single Operand Instructions

- 7 single operand instructions:

Mnemonic	Operation	Description
Logical and register control instructions		
RRA (.B or .W) dst	MSB→MSB→... LSB→C	Roll destination right
RRC (.B or .W) dst	C→MSB→...LSB→C	Roll destination right through carry
SWPB (or .W) dst	Swap bytes	Swap bytes in destination
SXT dst	bit 7→bit 8...bit 15	Sign extend destination
PUSH (.B or .W) src	SP-2→SP, src→@SP	Push source on stack
Program flow control instructions		
CALL (.B or .W) dst	SP-2→SP, PC+2→@SP dst→PC	Subroutine call to destination
RETI	@SP+→SR, @SP+→SP	Return from interrupt

MSP430 CPU ISA - Instructions

Example: Single Operand

- Logically shift the contents of register **r5** to the right through the status register carry
 - Assembly: **rrc.w r5**
 - Instruction code: **0x1005**

<u>Op-code</u> <i>rrc</i>	<u>b/w</u> <i>16-bits</i>	<u>Ad</u> <i>Register</i>	<u>D-reg</u> <i>r5</i>
0 0 0 1 0 0 0 0 0	0	0 0	0 1 0 1

- One word instruction
- The CPU shifts the 16-bit register **r5** one bit to the right (divide by 2) – the carry bit prior to the instruction becomes the MSB of the result while the LSB shifted out replaces the carry bit in the status register

MSP430 CPU ISA - Instructions

Format II: Jump Instructions

- These 8 jump instructions are used to direct program flow to another part of the program.

Mnemonic	Description
Program flow control instructions	
JEQ/JZ label	Jump to label if zero flag is set
JNE/JNZ label	Jump to label if zero flag is reset
JC label	Jump to label if carry flag is set
JNC label	Jump to label if carry flag is reset
JN label	Jump to label if negative flag is set
JGE label	Jump to label if greater than or equal
JL label	Jump to label if less than
JMP label	Jump to label unconditionally

“label” is where the program will jump to if the conditions are met.

MSP430 CPU ISA - Instructions

Format III: Jump Instructions Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code					Condition			10-bit, 2's complement PC offset							

- Jump instructions are used to direct program flow to another part of the program.
- The condition on which a jump occurs depends on the Condition field consisting of 3 bits:
 - 000: jump if not equal ($Z = 0$)
 - 001: jump if equal ($Z = 1$)
 - 010: jump if carry flag equal to zero ($C = 0$)
 - 011: jump if carry flag equal to one ($C = 1$)
 - 100: jump if negative ($N = 1$)
 - 101: jump if greater than or equal ($N = V$)
 - 110: jump if lower ($N \neq V$)
 - 111: unconditional jump

MSP430 CPU ISA - Instructions

Format III: Jump Instructions Format

- Jump instructions are executed based on the current PC and the status register
- Conditional jumps are controlled by the status bits
- Status bits are not changed by a jump instruction
- The jump off-set is represented by the 10-bit, 2's complement value:

$$PC_{new} = PC_{old} + 2 + PC_{offset} \times 2$$

PC_{old} is the address of the Jump instruction

- Thus, the range of the jump is -511 to +512 words, (-1022 to 1024 bytes) from the current instruction

MSP430 CPU ISA - Instructions

Example: Jump Instruction

- Continue execution at the label **Delay** if the Zero bit (Z-flag) is not set
 - Assembly: **jnz Delay**
 - Instruction code: **0x23FE**

<u>Op-code</u> <u>Jump</u>	<u>Condition</u> <u>Not Zero</u>	<u>10-Bit, 2's complement PC offset</u> <u>-2</u>
0 0 1	0 0 0	1 1 1 1 1 1 1 1 1 0

0	0	1	condition			10-bit signed offset				
0	0	1	0	0	0	10-bit signed offset				JNE/JNZ Jump if not equal/zero

- One word instruction
- The CPU will add to the incremented **PC** (R0) the value **-2 x 2** if the zero flag is not set (i.e, Z=0)

Chapter Contents



<https://ict1011.stevenwong.org/c3t7/>

1. Examples of Computers

- Micro-processor based and Micro-controller based systems
- The MSP430 MCU Overview

2. MSP430 CPU ISA

- CPU Organisation
- Registers
- Memory Organisation
- Machine Code & Assembly Instructions
- Addressing Modes

Addressing Modes Overview

<u>Op-code</u> <i>mov</i>	<u>S-reg</u> <i>r5</i>	<u>Ad</u> <i>Register</i>	<u>b/w</u> <i>16-bits</i>	<u>As</u> <i>Register</i>	<u>D-reg</u> <i>r4</i>
0 1 0 0	0 1 0 1	0	0	0 0	0 1 0 0

- Opcodes tells the ALU what operations to perform, but the addressing modes specifies the source and destination of the parameters (or operands).
- There are seven addressing modes that take a source operand and four of these addressing modes also take a destination operand;
- The operands can be located in any memory space address, but the user needs to be aware of the effects that their accesses may have;
- The addressing modes are selected by the A_S and A_D fields that make up the instruction

Addressing Modes Overview

<u>Op-code</u> <i>add</i>	<u>S-reg</u> <i>r1</i>	<u>Ad</u> <i>Register</i>	<u>b/w</u> <i>16-bits</i>	<u>As</u> <i>Register</i>	<u>D-reg</u> <i>r10</i>
0 1 0 1	0 0 0 1	0	0	0 0	1 0 1 0

Operands (single-operand instructions)

Source operands (double-operand instructions)

Addressing mode	As	S-reg
Register mode	0 0	0 0 0 0 to 1 1 1 1
Indexed mode	0 1	0 0 0 1, 0 0 1 1 to 1 1 1 1
Symbolic mode	0 1	0 0 0 0
Absolute mode	0 1	0 0 1 0
Indirect register mode	1 0	0 0 0 0 to 1 1 1 1
Indirect auto increment mode	1 1	0 0 0 1 to 1 1 1 1
Immediate mode	1 1	0 0 0 0

Destination operands (double-operand instructions)

Addressing mode	Ad	D-reg
Register mode	0	0 0 0 0 to 1 1 1 1
Indexed mode	1	0 0 0 1, 0 0 1 1 to 1 1 1 1
Symbolic mode	1	0 0 0 0
Absolute mode	1	0 0 1 0
Indirect register mode		
Indirect auto increment mode		
Immediate mode		

MSP430 CPU ISA – Addressing Modes

Source Operand Mnemonics

- The MSP430 has four modes for the source address:
 - Rs - Register *ADD.W R4, R10*
 - x(Rs) - Indexed Register *ADD.W 6(R4), R10*
 - @Rs - Register Indirect *ADD.W @R4, R10*
 - @Rs+ - Indirect Auto-increment *ADD.W @R4+, R10*
- In combination with registers R0/R2, three additional source addressing modes are available:
 - label - PC Relative, x(PC) *ADD.W cnt, R10*
 - &label - Absolute, x(SR) *ADD.W &cnt, R10*
 - #n - Immediate, @PC+ *ADD.W #100, R10*

MSP430 CPU ISA – Addressing Modes

Destination Operand Mnemonics

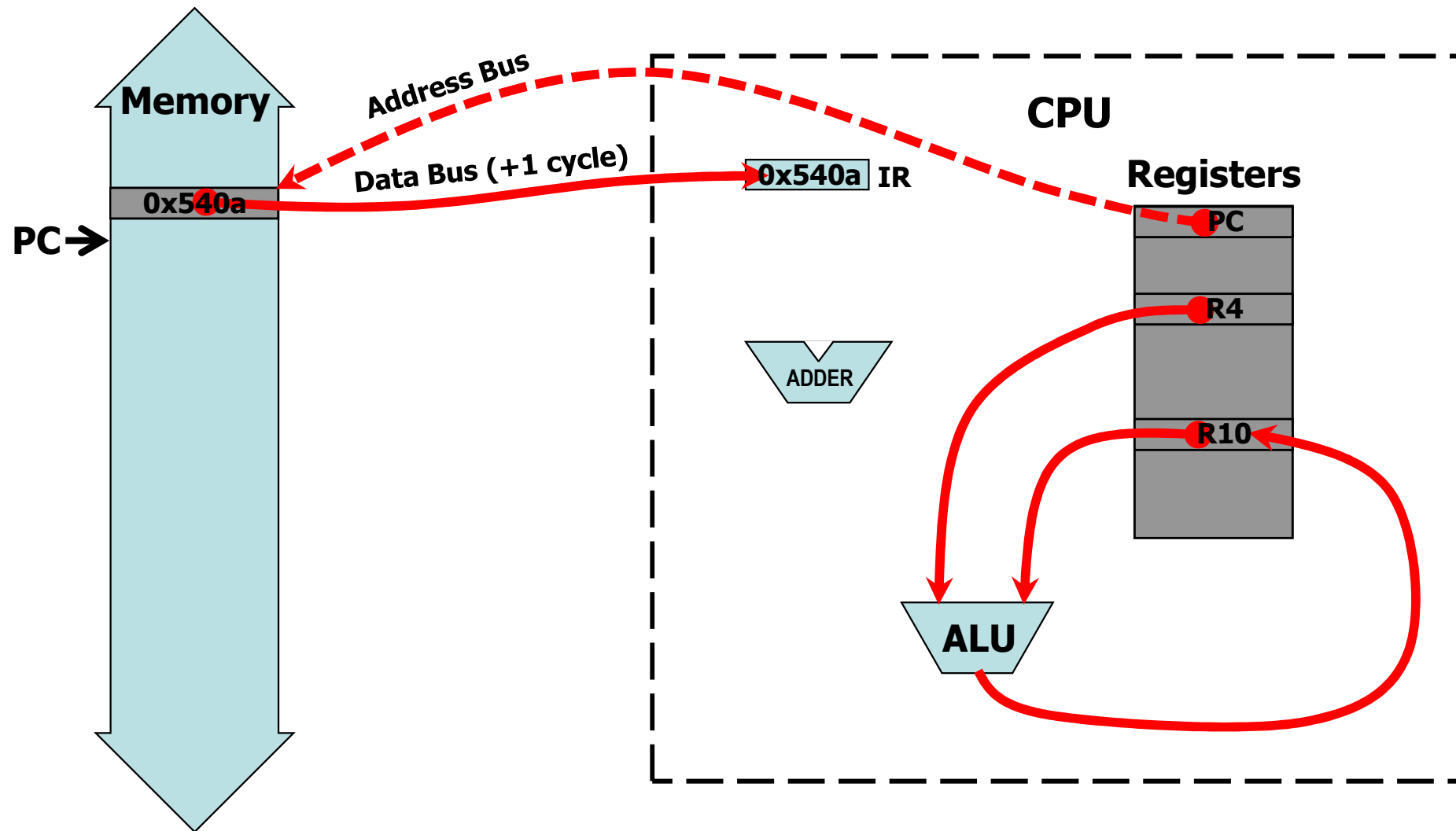
- There are two modes for the destination address:
 - 0 = Rd - Register *ADD.W R4, R10*
 - 1 = x(Rd) - Indexed Register *ADD.W R4, 6(R10)*
- In combination with registers R0/R2, two additional destination addressing modes are available:
 - label - PC Relative, x(PC) *ADD.W R4, cnt*
 - &label – Absolute, x(SR) *ADD.W R4, &cnt*

MSP430 CPU ISA – Addressing Mode 1

Register Mode – Example 1

`add.w r4, r10`

`; r10 = r4 + r10`



MSP430 CPU ISA – Addressing Mode 1

Register Mode

- In Register addressing mode, the contents of a register is used as the operand;
- This type of addressing mode can be used both with source and destination operands.

- Add the contents of register R4 to register R10:

ADD R4, R10

– Instruction code: 0x540a

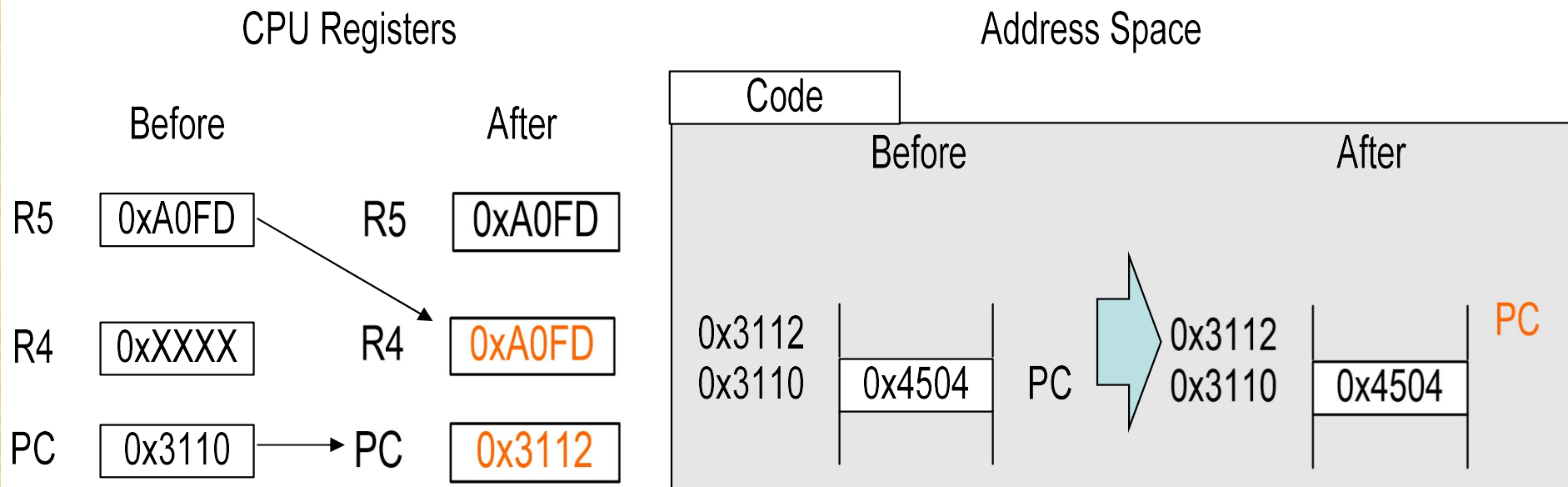
Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 1	0 1 0 0	0	0	0 0	1 0 1 0
ADD	R4	Register	16-bit	Register	R10

MSP430 CPU ISA – Addressing Mode 1

Register Mode – Example 2

- Move the contents of register R5 to register R4:
MOV R5, R4
– Instruction code: 0x4504

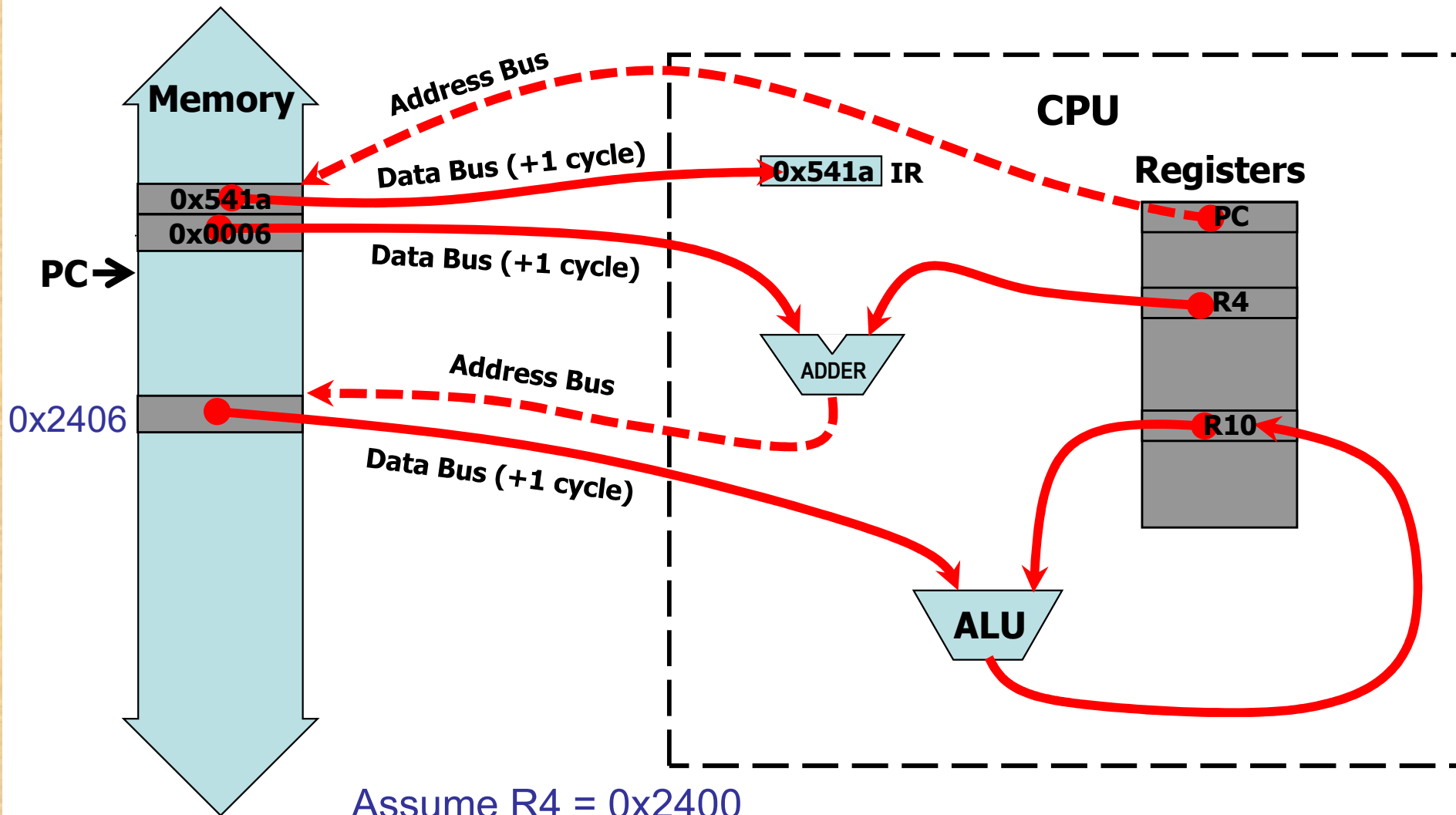
Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 1 0 1	0	0	0 0	0 1 0 0
MOV	R5	Register	16-bit	Register	R4



MSP430 CPU ISA – Addressing Mode 2

Indexed Mode – Example 1

`add.w 6(r4), r10 ; r10 = M(r4+6) + r10`



Assume $R4 = 0x2400$

Address of source data = $R4 + 6 = 0x2406$

MSP430 CPU ISA – Addressing Mode 2

Indexed Mode

- In indexed mode, whether used to indicate the source address or the destination address, the sum of the register contents and the signed offset point to the location in memory;
- The offset value is stored in the word following the instruction;
- After the execution of the instruction, the contents of registers are not affected and the PC is incremented to point to the next instruction to be executed;
- This addressing mode is useful to access data stored in tables. Apart from the registers R0 (PC), R2 (SR) and R3 all other registers can be used as an index in indexed mode.

MSP430 CPU ISA – Addressing Mode 2

Indexed Mode – Example 2

- Move the byte pointed to by (R5 + 4) to the byte pointed to by (R4 + 1):

MOV.B 4 (R5) , 1 (R4)

– Instruction code: 0x45D4

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 1 0 1	1	1	0 1	0 1 0 0
MOV	R5	Indexed	8-bit	Indexed	R4

- The instruction coding specifies that the byte (***B/W = 1***) pointed to by the sum of the contents of register R5 (***S-reg = 0101***) and the word ***x1*** should be moved to the memory address pointed to by the sum of register R4 (***D-reg = 0100***) and the contents of the word ***x2***;

MSP430 CPU ISA – Addressing Mode 2

Indexed Mode – Example 2

- Move the byte pointed to by (R5 + 4) to the byte pointed to by (R4 + 1) (continued):

MOV.B 4 (R5) , 1 (R4)

– Instruction code: 0x45D4 0x0004 0x0001

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 1 0 1	1	1	0 1	0 1 0 0
MOV	R5	Indexed	8-bit	Indexed	R4

- The words **x1** and **x2** are located in the memory addresses following the instruction;
- The addressing mode used for the source and destination operands is controlled by the bits **Ad = 1** (Indexed mode) and **As = 01** (Indexed mode).

MSP430 CPU ISA – Addressing Mode 2

Indexed Mode – Example 2

- Move the byte pointed to by (R5 + 4) to the byte pointed to by (R4 + 1) (continued):

MOV.B 4(R5), 1(R4)

CPU Registers

	Before		After
R5	0x0200	R5	0x0200
R4	0x0200	R4	0x0200
PC	0x3110	PC	0x3116

Address Space

Code	Before		After
0x3116			0x3116
0x3114	0x0001	X2	0x3114
0x3112	0x0004	X1	0x3112
0x3110	0x45D4	PC	0x3110

Destination Address

$$\begin{array}{r}
 0x0200 \quad (R4) \\
 + 0x0001 \quad (X2) \\
 \hline
 0x0201
 \end{array}$$

Source Address

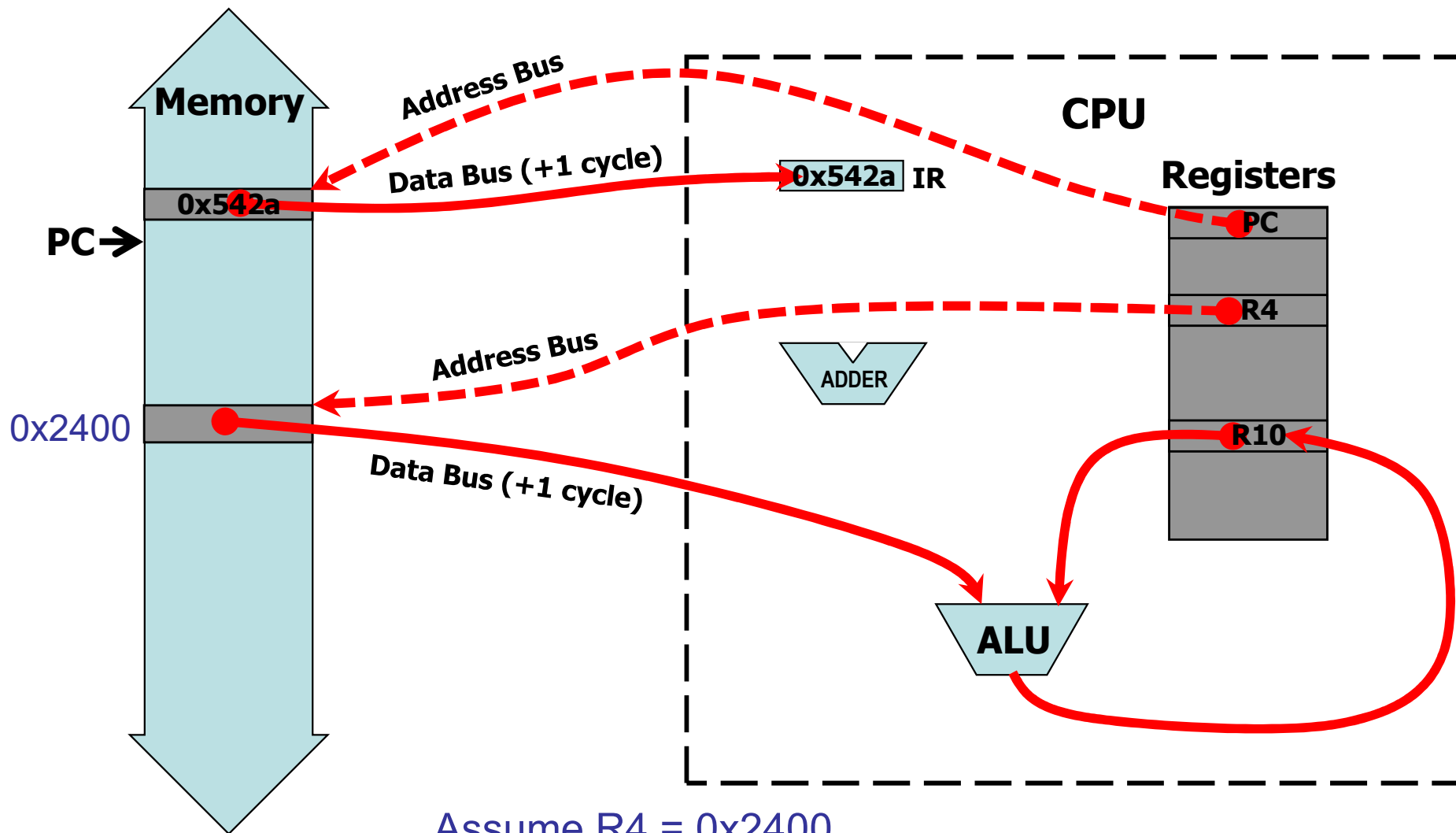
$$\begin{array}{r}
 0x0200 \quad (R5) \\
 + 0x0004 \quad (X1) \\
 \hline
 0x0204
 \end{array}$$

Data		
0x0206		0x0206
0x0204	0x9ABC	0x0204
0x0202	0x5678	0x0202
0x0200	0x1234	0x0200
0x0206		
0x0204	0x9ABC	
0x0202	0x5678	
0x0200	0x1234	

MSP430 CPU ISA – Addressing Mode 3

Indirect Register Mode – Example 1

`add.w @r4, r10` ; $r10 = M(r4) + r10$



MSP430 CPU ISA – Addressing Mode 3

Indirect Register Mode

- In Indirect register mode, any of the CPU registers can be used.
- A restriction arises from the fact that this addressing mode *can only be used to specify the source operand address* in dual-operand instructions;
- An alternative way to avoid this restriction is to use indexed mode to specify the destination operand address, with a zero offset.

MOV R10, @R5 X(Not Supported) → MOV R10, 0(R5)

Indirect Register Mode – Example 2

- Move the word pointed to by R5 to the word pointed to by R4:

MOV @R5, 0 (R4)

– Instruction code: 0x45A4 0x0000

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 1 0 1	1	0	1 0	0 1 0 0
MOV	R5	Indexed	16-bit	Indirect	R4

- The instruction coding specifies that register R5 (*S-reg = 0101*) uses the source address (*As = 10*);
- The destination address is pointed to in Indexed mode (*Ad = 1*) by R4 (*D-reg = 0100*), using a zero value offset.

MSP430 CPU ISA – Addressing Mode 3

- Move the word pointed to by R5 to the word pointed to by R4 (continued):

MOV @R5, 0 (R4)

CPU Registers			
	Before		After
R5	0x0200	R5	0x0200
R4	0x0202	R4	0x0202
PC	0x3110	PC	0x3114

Code		Address Space	
Before		After	
0x3114		0x3114	
0x3112	0x0000	0x3112	0x0000
0x3110	0x45A4	0x3110	0x45A4

$$\begin{array}{r} \text{Destination Address} \\ 0x0202 \quad (\text{R4}) \\ + 0x0000 \quad (\text{X1}) \\ \hline 0x0202 \end{array}$$

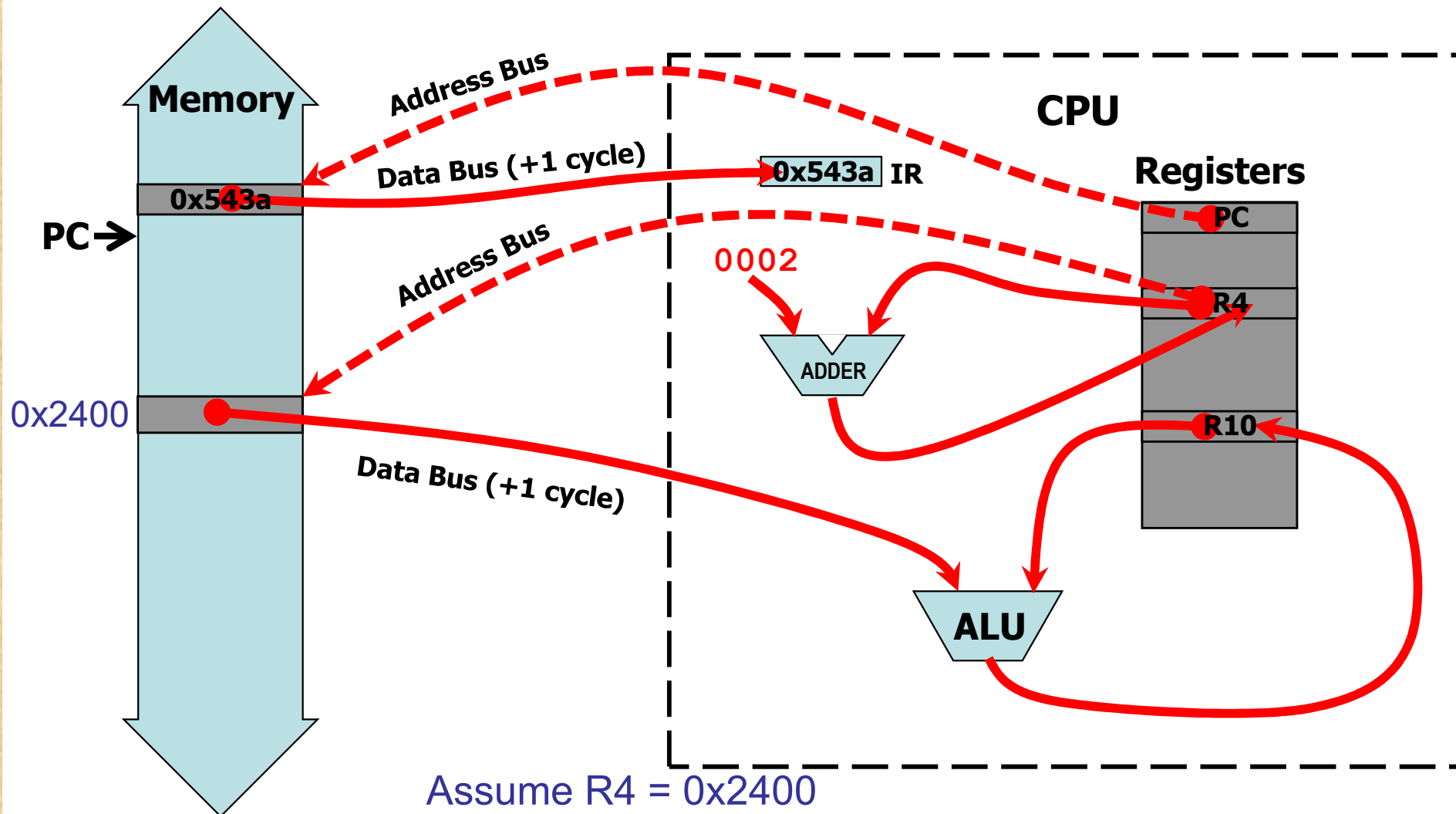
Source Address

Data					
0x0206		0(R4)	0x0206		0(R4)
0x0204	0x9ABC		0x0204	0x9ABC	
0x0202	0x5678		0x0202	0x1234	
0x0200	0x1234		0x0200	0x1234	
0x0206		@R5			
0x0204	0x9ABC				
0x0202	0x5678				
0x0200	0x1234				

MSP430 CPU ISA – Addressing Mode 4

Indirect Auto-inc Mode – Example 1

`add.w @r4+, r10 ; r10 = M(r4+) + r10`



Assume R4 = 0x2400
R4 = 0x2402 after the read from memory

Chpt 3: Instruction Set Architecture

MSP430 CPU ISA – Addressing Mode 4

Indirect Auto-increment Mode

- This addressing mode is similar to the previous one;
- The contents of source register is incremented according to the data type processed;
- If the *data value is byte*, the source register is *incremented by 1*;
- If the *data value is a word*, the source register is *incremented by 2*;
- Note that this addressing mode can only be used to specify the source operand in dual-operand instructions.

Indirect Auto-inc Mode – Example 2

- Move the word pointed to by R5 to the word pointed to by R4, and increment the source pointer:

MOV @R5+, 0 (R4)

– Instruction code: 0x45B4 0x0000

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 1 0 1	1	0	1 1	0 1 0 0
MOV	R5	Indexed	16-bit	Ind. aut. inc.	R4

- The instruction coding specifies that the register R5 (*S-reg = 0101*) contains the source address (*As = 11*);
- The destination address is pointed to in indexed mode by R4 (*D-reg = 0100*), using a zero value offset;
- The execution of the instruction **increments the contents of register R5 by 2**.

MSP430 CPU ISA – Addressing Mode 4

- Move the word pointed to by R5 to the word pointed to by R4, and increment the source pointer (continued):

MOV @R5+, 0 (R4)

CPU Registers

Before		After	
R5	0x0200	R5	0x0202
R4	0x0202		
PC	0x3110	PC	0x3114

$$\begin{array}{r} \text{Destination Address} \\ 0x0202 \quad (\text{R4}) \\ + 0x0000 \quad (\text{X1}) \\ \hline 0x0202 \end{array}$$

Source Address

Try this yourself!!!
MOV @R5+, 0(R5)

Code		Address Space	
Before		After	
0x3114		←	0x3114
0x3112	0x0000	X1 ←	0x3112
0x3110	0x45A4	PC ←	0x3110

Data	
0x0206	
0x0204	0x9ABC
0x0202	0x5678
0x0200	0x1234

0(R4)

0x0206	
0x0204	0x9ABC
0x0202	0x1234
0x0200	0x1234

0(R4)

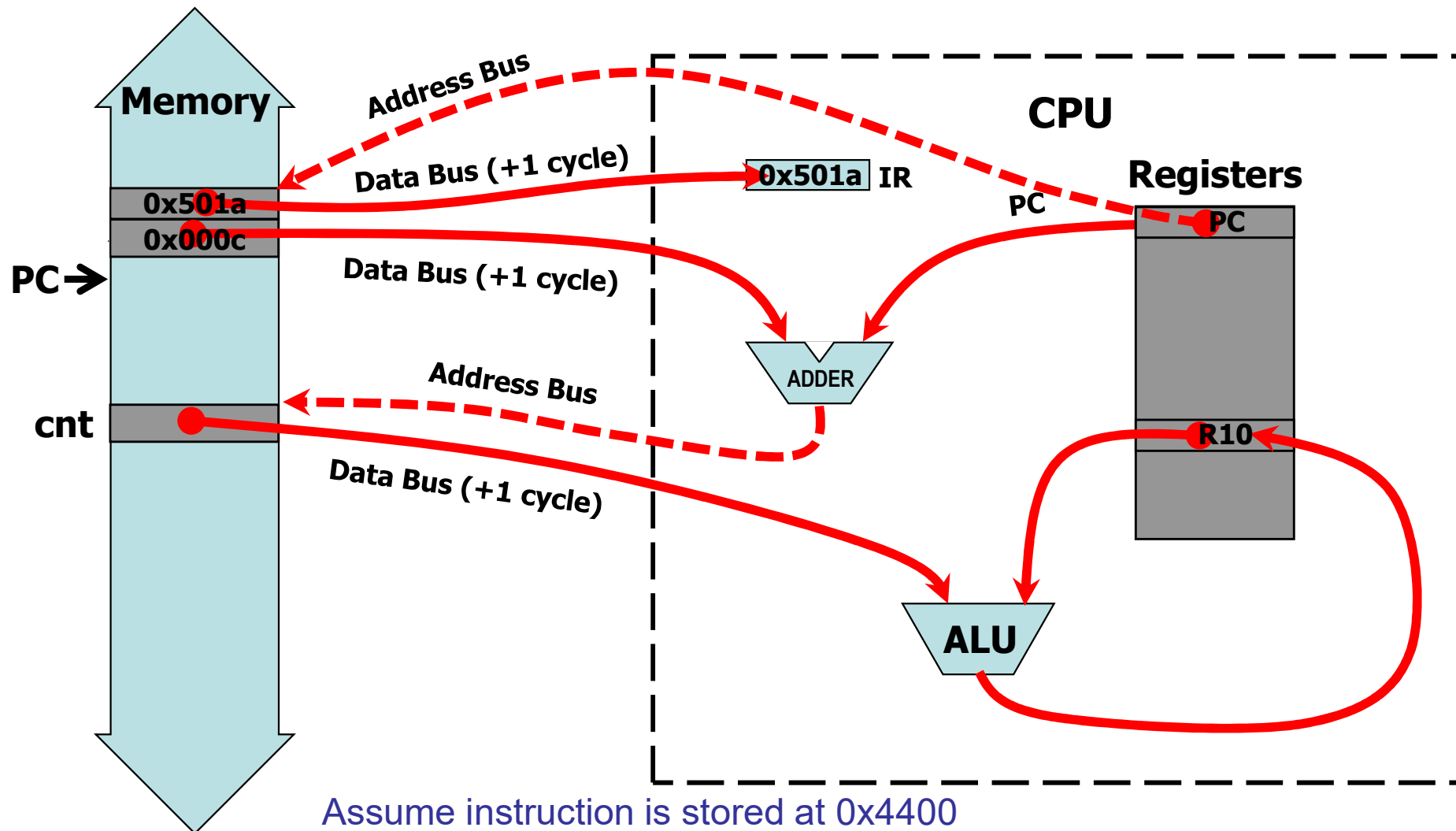
0x0206	
0x0204	0x9ABC
0x0202	0x5678
0x0200	0x1234

@R5

MSP430 CPU ISA – Addressing Mode 5

Symbolic Mode (PC-relative) – Example 1

`add.w cnt, r10 ; r10 = M(cnt) + r10`



Assume instruction is stored at 0x4400
 The label **cnt** is stored at location 0x440E
 Chpt 3: Instruction Set Architecture

MSP430 CPU ISA – Addressing Mode 5

Symbolic Mode (PC-relative)

- In symbolic addressing mode, for either the source or destination operands, the memory address is determined by adding an offset to the program counter (PC) register;
- The offset value is obtained by determining the code position in the memory, then calculating the difference between the code position and the memory position that should be achieved;
- The assembler determines the offset value and puts it in the word following to the instruction;
- After the execution of the instruction, the program counter (PC) register is incremented in order to point to the next instruction.

Symbolic Mode (PC-relative) – Example 2

- Move the word pointed to by EDEN to the word pointed to by TONI:

```
MOV EDEN, TONI
```

– Instruction code: 0x4090 0x---- 0x----

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 0 0 0	1	0	0 1	0 0 0 0
MOV	PC	Symbolic	16-bit	Symbolic	PC

- The instruction coding specifies that the value pointed to by the sum of the PC register contents (*S-reg = 0*) and the word *x1* should be moved to the memory address pointed to by the sum of register PC contents (*D-reg = 0*) and the word *x2*;
- The words *x1* and *x2* are stored in the memory addresses following the instruction.

Symbolic Mode (PC-relative) – Example 2

- Move the word pointed to by EDEN to the word pointed to by TONI (continued) :

```
MOV EDEN, TONI
```

– Instruction code: 0x4090

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 0 0 0	1	0	0 1	0 0 0 0
MOV	PC	Symbolic	16-bit	Symbolic	PC

- The addressing mode used for the source and destination operands is controlled by the bits **Ad = 1** (Symbolic mode) and **As = 01** (Symbolic mode).

Symbolic Mode (PC-relative) – Example 2

- Move the word pointed to by EDEN to the word pointed to by TONI:

MOV EDEN, TONI

CPU Registers

Before		After	
PC	0x3110	PC	0x3116

Address Space

Code		Before		After	
0x3116				0x3116	PC
0x3114	0xD0EE	X2 ←		0x3114	0xD0EE X2
0x3112	0xD0EE	X1 ←		0x3112	0xD0EE X1
0x3110	0x4090	PC ←		0x3110	0x4090

Destination Address

$$\begin{array}{r}
 0x3114 \quad (\text{PC}) \\
 +0xD0EE \quad (\text{X2}) \\
 \hline
 0x0202
 \end{array}$$

Source Address

$$\begin{array}{r}
 0x3112 \quad (\text{PC}) \\
 +0xD0EE \quad (\text{X1}) \\
 \hline
 0x0200
 \end{array}$$

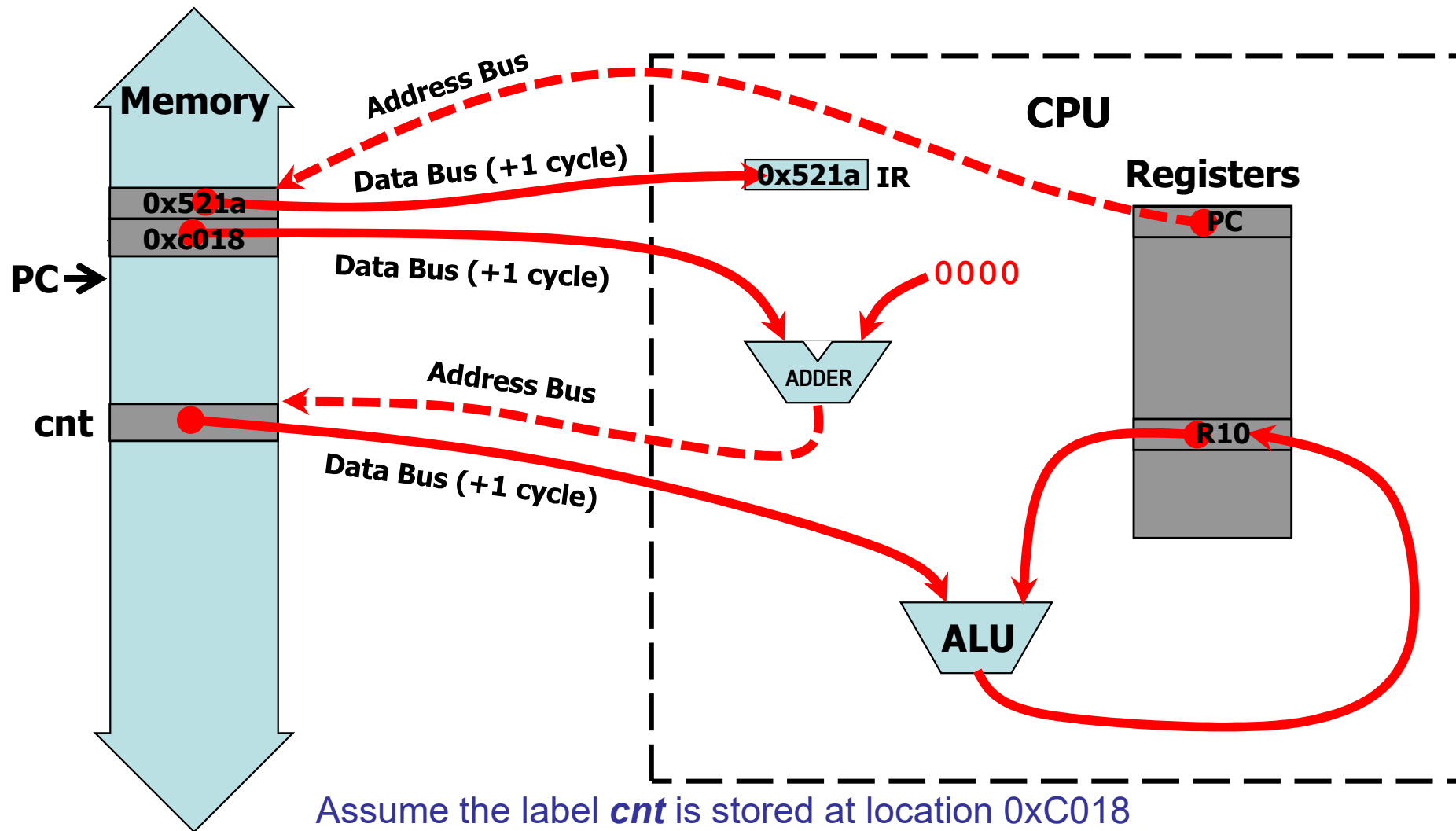
Data		Before		After	
0x0206				0x0206	
0x0204	0x9ABC			0x0204	0x9ABC
0x0202	0x5678			0x0202	0x1234 TONI
0x0200	0x1234			0x0200	0x1234

0x0206					
0x0204	0x9ABC				
0x0202	0x5678				
0x0200	0x1234	EDEN			

MSP430 CPU ISA – Addressing Mode 6

Absolute Addressing Mode – Example 1

`add.w &cnt, r10 ; r10 = M(cnt) + r10`



MSP430 CPU ISA – Addressing Mode 6

Absolute Addressing Mode

- In Absolute mode, the data memory address is placed after the instruction;
- The difference between this addressing mode and Indexed mode is that the register R2 is now used as an index, using the constants generator to generate the value zero;
- This addressing mode can be used to define either the source address or the destination address.

Absolute Addressing Mode – Example 2

- Move the word pointed to by EDEN to the word pointed to by TONI:

MOV &EDEN, &TONI

– Instruction code: 0x4292 0x---- 0x----

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 0 1 0	1	0	0 1	0 0 1 0
MOV	R2/CG1	Absolute	16-bit	Absolute	R2/CG1

- It can be seen that the register R2 has (*S-reg = 0010*) and (*D-reg = 0010*). Register R2 is used as an addresses index, in which the constant generator loads the value zero;
- When the contents of this register is added to the offset value *x1* or *x2*, the source and destination addresses are obtained;
- The words *x1* and *x2* are stored in the memory addresses following the instruction.

Absolute Addressing Mode – Example 2

- Move the word pointed to by EDEN to the word pointed to by TONI (continued):

MOV &EDEN, &TONI

– Instruction code: 0x4292

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 0 1 0	1	0	0 1	0 0 1 0
MOV	R2/CG1	Absolute	16-bit	Absolute	R2/CG1

- The addressing mode used for the source and destination operands is specified by the bits **Ad = 1** (Absolute mode) and **As = 01** (Absolute mode).

Absolute Addressing Mode – Example 2

- Move the word pointed to by EDEN to the word pointed to by TONI (continued):

MOV &EDEN, &TONI

CPU Registers

	Before	After
PC	0x3110	0x3116

Address Space

Code	Before	After
0x3116		PC
0x3114	0x0202	X2
0x3112	0x0200	X1
0x3110	0x4292	

Destination Address

$$\begin{array}{rcl}
 0x0000 & (R2) \\
 + 0x0202 & (X2) \\
 \hline
 0x0202
 \end{array}$$

Source Address

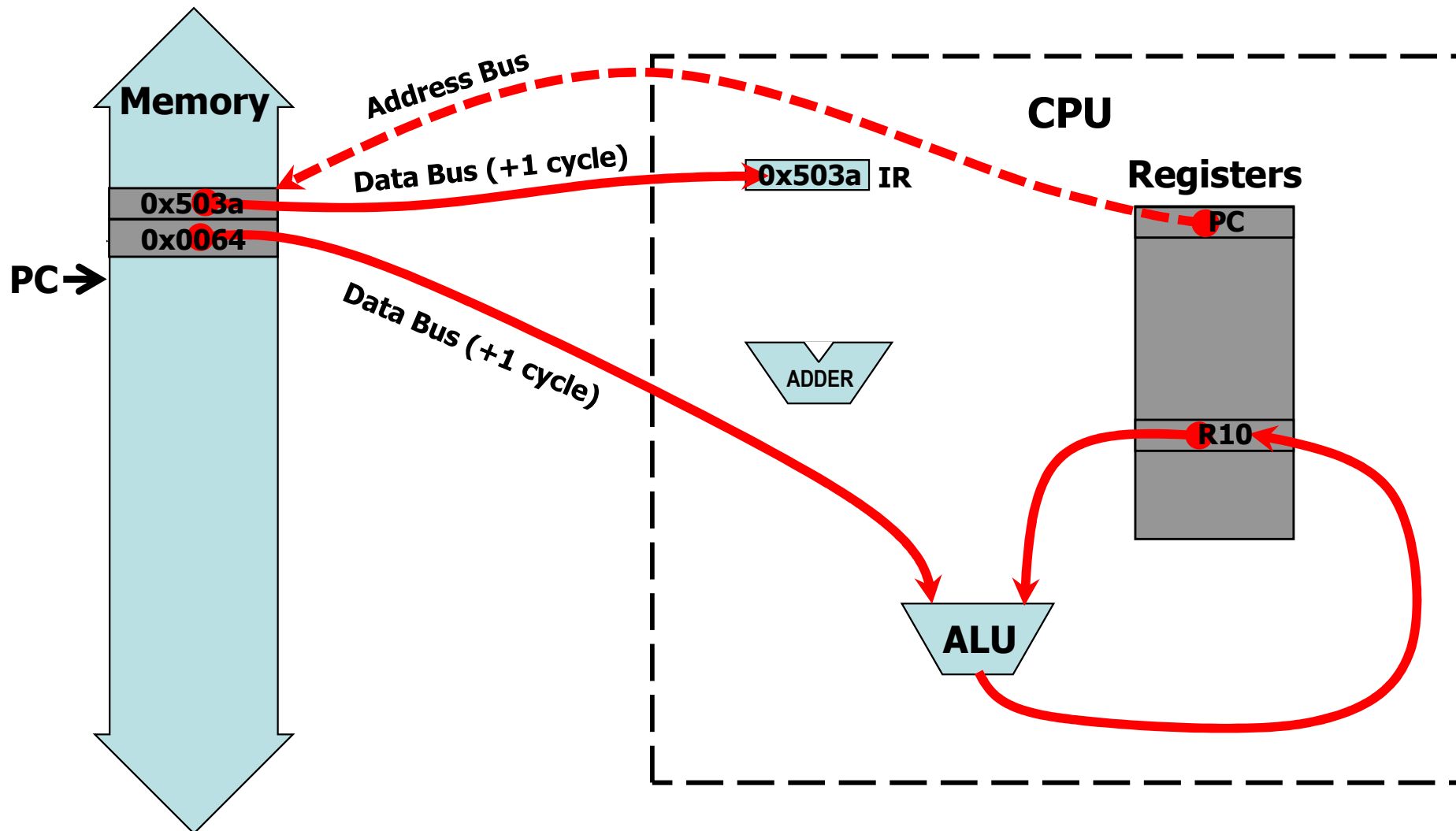
$$\begin{array}{rcl}
 0x0000 & (R2) \\
 + 0x0200 & (X1) \\
 \hline
 0x0200
 \end{array}$$

Data	
0x0206	
0x0204	0x9ABC
0x0202	0x5678
0x0200	0x1234
	TONI
0x0206	
0x0204	0x9ABC
0x0202	0x1234
0x0200	0x1234
	EDEN

MSP430 CPU ISA – Addressing Mode 7

Immediate Mode – Example 1

`add.w #100, r10 ; r10 = #100 + r10`



MSP430 CPU ISA – Addressing Mode 7

Immediate Mode

- The Immediate addressing mode allows values to be loaded directly into registers or memory addresses. It can only be used with source operands.
- Move the value 0x0200 to R5:

MOV #0x0200, R5

– Instruction code: 0x4035 **0x0200**

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 0	0 0 0 0	0	0	1 1	0 1 0 1
MOV	PC	Register	16-bit	Immediate	R5

- The instruction coding specifies that the register PC (***S-reg = 0000***) is used to define the address of the word in memory that loads the register **R5** (***D-reg = 0101***) with (***As = 11***).

MSP430 CPU ISA – Addressing Mode 7

Immediate Mode – Example 3

- Move the value 0x0200 to R5 (continued):

```
MOV #0x0200, R5
```

CPU Registers

	Before		After
R5	0XXXXX	R5	0x0200
PC	0x3110	PC	0x3114

Address Space

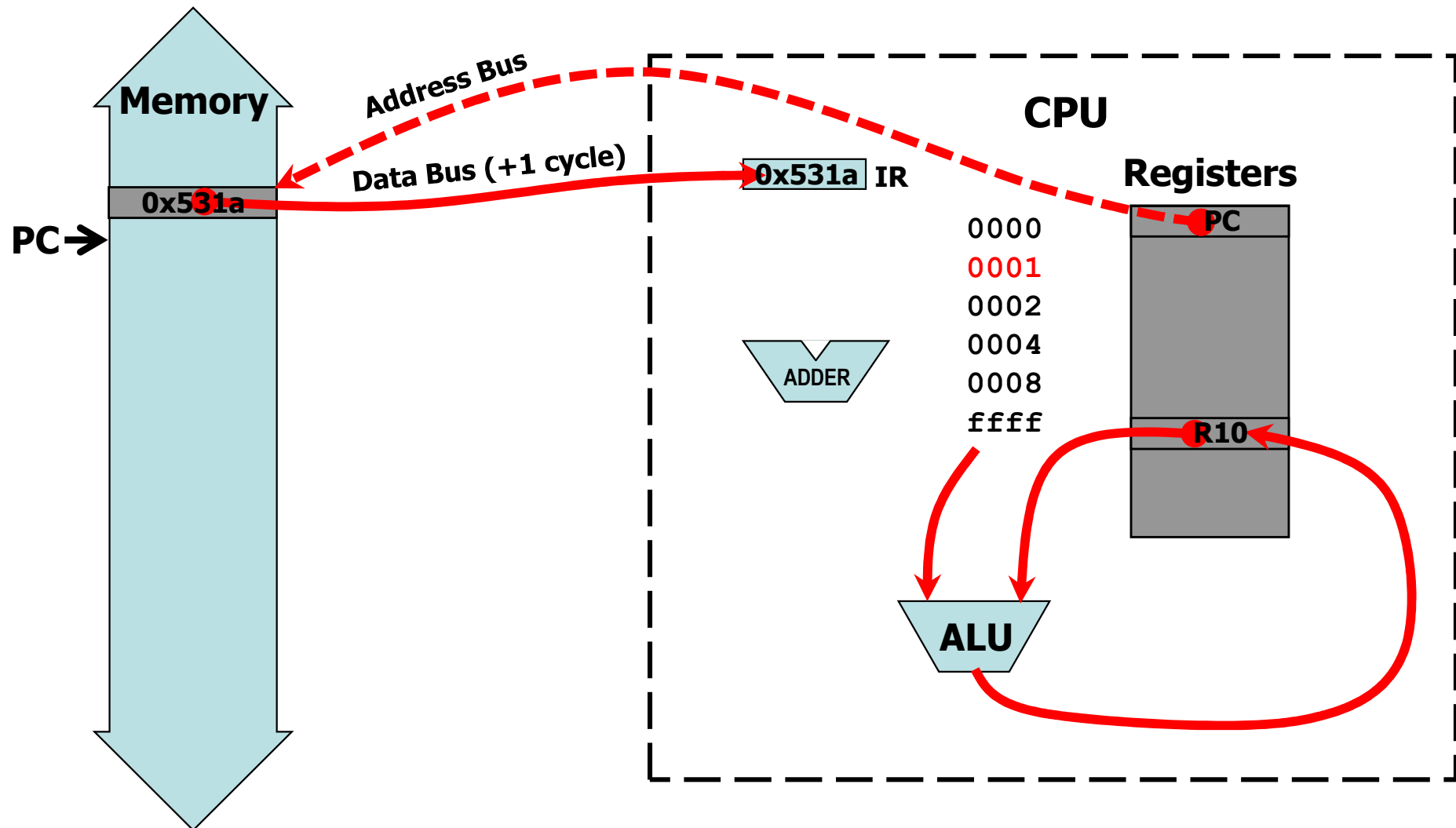
Code				Before		After	
0x3114				0x3114		PC	
0x3112	0x0200	X1		0x3112	0x0200	X1	
0x3110	0x4035	PC		0x3110	0x4035		

MSP430 CPU ISA – Addressing Mode 7

Immediate Mode – Example 2

`add.w #1, r10` ; $r10 = \#1 + r10$

Op-code	S-reg	Ad	B/W	As	D-reg
0 1 0 1	0 0 1 1	0	0	0 1	1 0 1 0
ADD	R3	Register	16-bit	Register	R10



Confused???



DON'T WORRY!!!

(still not the time for that yet)

**Try out some of the stuff in
this lecture on your
Lauchpad!!!**



Blurr???

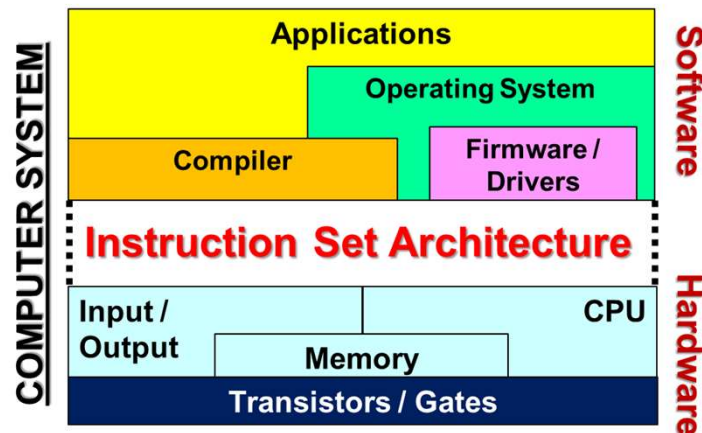
Summary



<https://ict1011.stevenwong.org/c3s/>

LEARNING OUTCOMES OF CHAPTER:

- ✓ Understand the ISA of the MSP430
- ✓ Understand how machine code instructions are constructed
- ✓ Understand and be able to use the various addressing modes available in the MSP430



NEXT UP:
The fun of coding
in Assembly
continues!!!